

Advanced Machine Learning

Lecture 12: Attention & Transformer

Jens-Peter M. Zemke
zemke@tuhh.de

Institut für Mathematik
Lehrstuhl Numerische Mathematik
Technische Universität Hamburg

21 January 2025



Outline

Repetition

- Autoencoder
- Variational autoencoder
- GAN
- WGAN

Attention & Transformer

- Attention
- Transformer
- BERT & GPT
- Vision transformer & multimodality

Outline

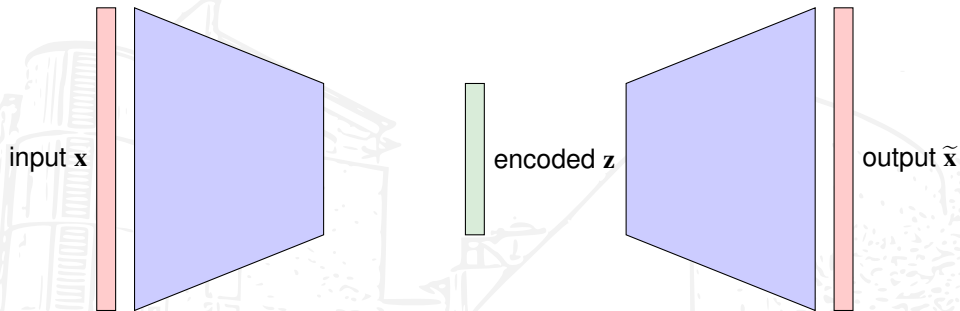
Repetition

- Autoencoder
- Variational autoencoder
- GAN
- WGAN

Attention & Transformer

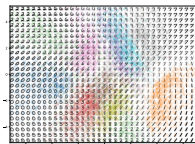
- Attention
- Transformer
- BERT & GPT
- Vision transformer & multimodality

Autoencoder



Autoencoder: reconstruct input x , latent representation z of lower dimension

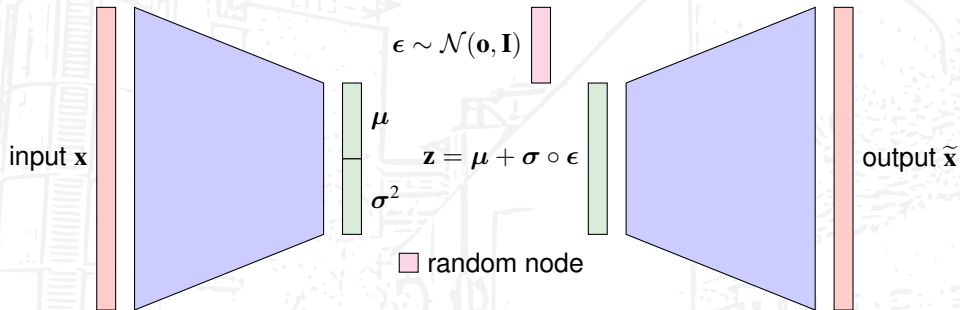
- ▶ linear autoencoder = PCA = SVD, Bourlard and Kamp [6]
- ▶ interpolation in latent space (distribution unknown)
- ▶ many applications (dimensionality reduction, denoising)



Variational autoencoder

Bregman divergences $\rightsquigarrow$ Kullback-Leibler divergence

$$D_F(\mathbf{p}, \mathbf{q}) := F(\mathbf{p}) - F(\mathbf{q}) - \langle \nabla F(\mathbf{q}), \mathbf{p} - \mathbf{q} \rangle, \quad D_{\text{KL}}(\mathbf{p} \parallel \mathbf{q}) := \mathbb{E}_{\mathbf{p}} \left[\log \left(\frac{p_i}{q_i} \right) \right].$$

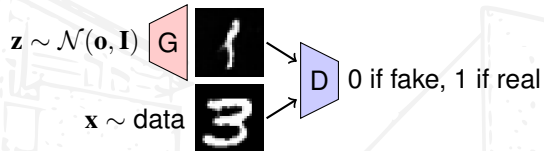


Cost function:

$$C_{\text{vae}}(\mathbf{x}) = \text{const} \cdot C_{\text{reconstruction}}(\mathbf{x}) + C_{\text{Kullback-Leibler}}(\mathcal{N}(\mu, \sigma^2), \mathcal{N}(\mathbf{o}, \mathbf{I})).$$

GAN

GAN (Goodfellow et al. [13], 2014):



Training of discriminator and generator w/ binary cross-entropy loss is **two-player minimax game**:

$$\min_{\mathbf{D}} \max_{\mathbf{G}} V(\mathbf{D}, \mathbf{G}) = \min_{\mathbf{D}} \max_{\mathbf{G}} \left(\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log \mathbf{D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - \mathbf{D}(\mathbf{G}(\mathbf{z})))] \right).$$

Nash equilibrium: $p_{\mathbf{G}} = p_{\text{data}}$, best measured by **Wasserstein- p distance**

$$W_p(\mu, \nu) := \left(\inf_{\gamma \in \Gamma(\mu, \nu)} \iint \|\mathbf{x} - \mathbf{y}\|_2^p \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} \right)^{1/p}, \quad \Gamma \text{ set of prob. m. w/ marginals } \mu, \nu.$$

WGAN (Arjovsky, Chintala, and Bottou [2], Gulrajani et al. [14])

Use **dual formulation** of **Wasserstein-1 (EMD)** to replace **discriminator** by “**critic**”:

$$W_1(\mu, \nu) = \inf_{\gamma \in \Gamma(\mu, \nu)} \iint \|\mathbf{x} - \mathbf{y}\| \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} = \sup_{\|f\|_L \leq 1} \mathbb{E}_{t \sim \mu}[f(t)] - \mathbb{E}_{t \sim \nu}[f(t)].$$

Wasserstein GAN (WGAN):

- ▶ model f by neural network w/ clamped weights
 $\rightsquigarrow$ Lipschitz w/ constant L (original idea by Arjovsky, Chintala, and Bottou [2])
- ▶ model f by neural network, add gradient penalty to cost function
 $\rightsquigarrow$ approximately Lipschitz w/ $L = 1$ (modified idea by Gulrajani et al. [14])

Gradient penalty: interpolate randomly between real and fake images, compute gradient norm

Critic loss: measures difference between outputs for real and fake

Weighted combination: used for training of critic

Outline

Repetition

- Autoencoder
- Variational autoencoder
- GAN
- WGAN

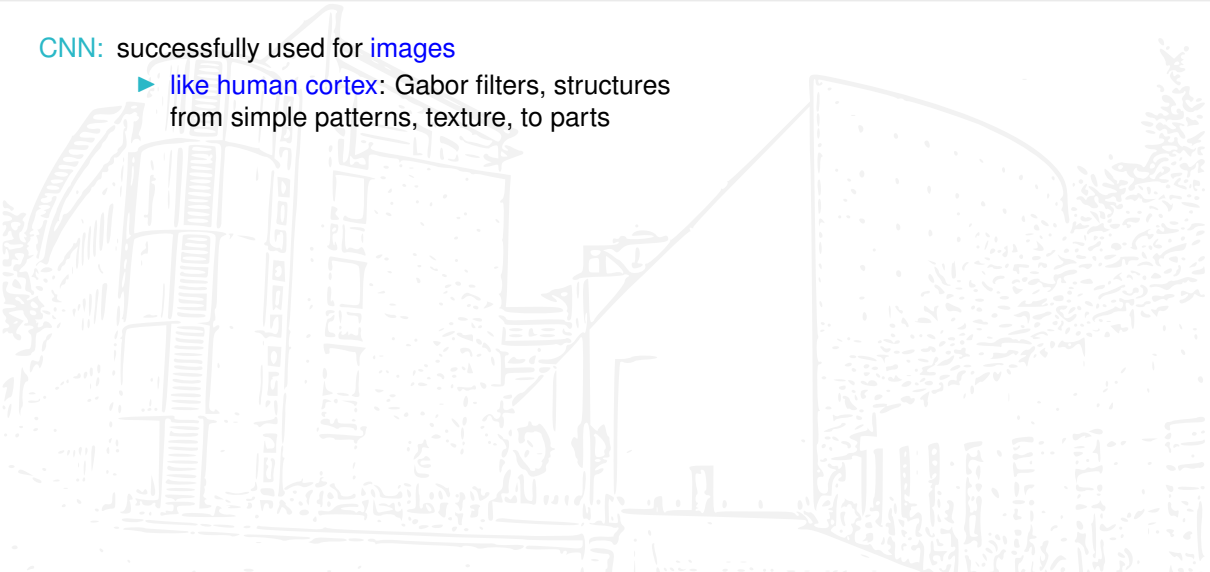
Attention & Transformer

- Attention
- Transformer
- BERT & GPT
- Vision transformer & multimodality

Motivation: you have to pay attention

CNN: successfully used for **images**

- ▶ **like human cortex**: Gabor filters, structures from simple patterns, texture, to parts



Motivation: you have to pay attention

CNN: successfully used for images

- ▶ like human cortex: Gabor filters, structures from simple patterns, texture, to parts
- ▶ but: no human would look at all regions of a picture with the same focus, e.g., when your “filter” detects a border, your eye follows it

Motivation: you have to pay attention

CNN: successfully used for **images**

- ▶ **like human cortex:** Gabor filters, structures from simple patterns, texture, to parts
- ▶ **but:** no human would look at all regions of a picture with the same **focus**, e.g., when your “filter” detects a border, your eye follows it

LSTM: successfully used for **language**

- ▶ **like natural language processing (NLP):** words come in sequences, different types of memory involved

Motivation: you have to pay attention

CNN: successfully used for **images**

- ▶ **like human cortex:** Gabor filters, structures from simple patterns, texture, to parts
- ▶ **but:** no human would look at all regions of a picture with the same **focus**, e.g., when your “filter” detects a border, your eye follows it

LSTM: successfully used for **language**

- ▶ **like natural language processing (NLP):** words come in sequences, different types of memory involved
- ▶ **but:** no human would **concentrate** on every word in a similar fashion (the, it, and, ...)

Motivation: you have to pay attention

CNN: successfully used for **images**

- ▶ **like human cortex:** Gabor filters, structures from simple patterns, texture, to parts
- ▶ **but:** no human would look at all regions of a picture with the same **focus**, e.g., when your “filter” detects a border, your eye follows it

LSTM: successfully used for **language**

- ▶ **like natural language processing (NLP):** words come in sequences, different types of memory involved
- ▶ **but:** no human would **concentrate** on every word in a similar fashion (the, it, and, ...)

Solution: data dependent weights, e.g., **attention!**

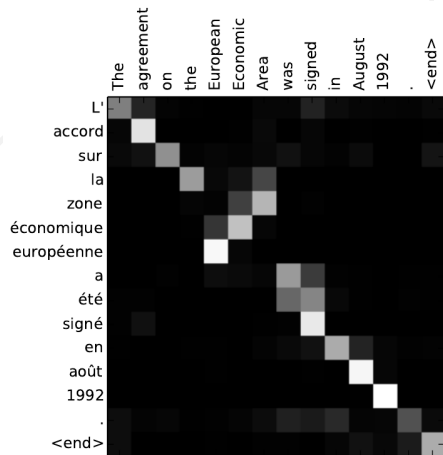
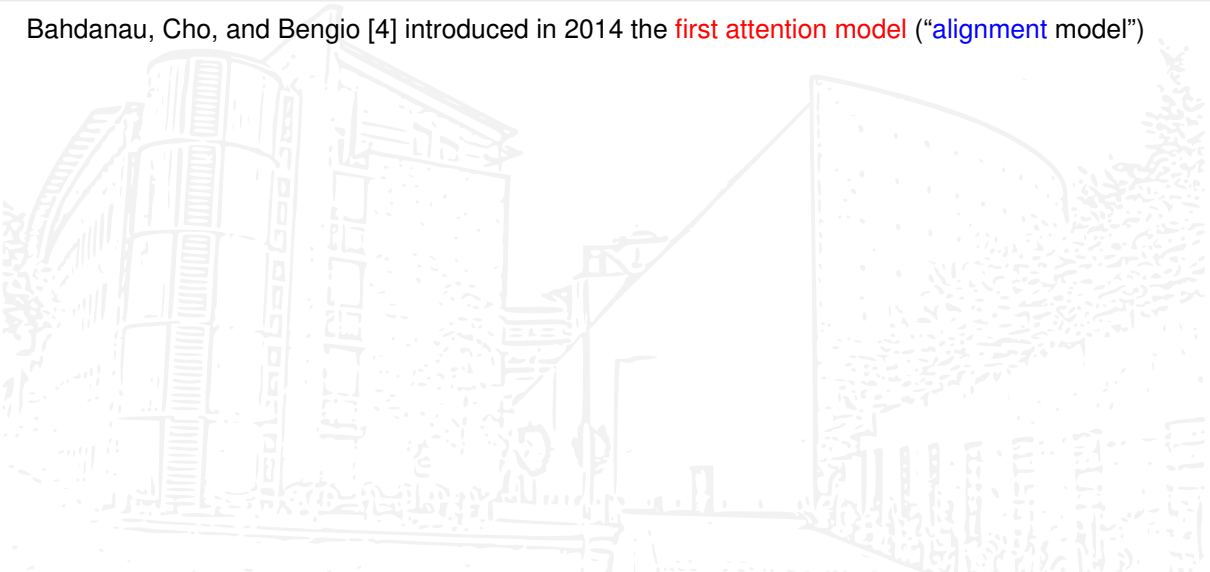


Figure taken from Bahdanau, Cho, and Bengio [4].

First work

Bahdanau, Cho, and Bengio [4] introduced in 2014 the **first attention model** (“**alignment** model”)



First work

Bahdanau, Cho, and Bengio [4] introduced in 2014 the **first attention model** (“**alignment model**”)

Most **neural machine translation (NMT)** models follow an **encoder-decoder** approach. **RNNSearch** by Bahdanau, Cho, and Bengio [4]: the **encoder** is a **bidirectional GRU**,

$$\{\vec{\mathbf{h}}_j\}_{j=1}^{T_{\text{in}}} = \overrightarrow{\text{GRU}}(\{\mathbf{x}_j\}_{j=1}^{T_{\text{in}}}), \quad \{\overleftarrow{\mathbf{h}}_j\}_{j=1}^{T_{\text{in}}} = \overleftarrow{\text{GRU}}(\{\mathbf{x}_j\}_{j=1}^{T_{\text{in}}}), \quad \mathbf{h}_j = \begin{pmatrix} \vec{\mathbf{h}}_j \\ \overleftarrow{\mathbf{h}}_j \end{pmatrix}$$

First work

Bahdanau, Cho, and Bengio [4] introduced in 2014 the **first attention model** (“**alignment model**”)

Most **neural machine translation (NMT)** models follow an **encoder-decoder** approach. **RNNSearch** by Bahdanau, Cho, and Bengio [4]: the **encoder** is a **bidirectional GRU**,

$$\{\vec{\mathbf{h}}_j\}_{j=1}^{T_{\text{in}}} = \overrightarrow{\text{GRU}}(\{\mathbf{x}_j\}_{j=1}^{T_{\text{in}}}), \quad \{\overleftarrow{\mathbf{h}}_j\}_{j=1}^{T_{\text{in}}} = \overleftarrow{\text{GRU}}(\{\mathbf{x}_j\}_{j=1}^{T_{\text{in}}}), \quad \mathbf{h}_j = \begin{pmatrix} \vec{\mathbf{h}}_j \\ \overleftarrow{\mathbf{h}}_j \end{pmatrix}$$

the **decoder** is based on the “**context vector**” $\mathbf{c}_i$

$$\mathbf{c}_i = \sum_{j=1}^{T_{\text{in}}} a_{i,j} \mathbf{h}_j, \quad a_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k=1}^{T_{\text{in}}} \exp(e_{i,k})},$$

$$e_{i,j} = \text{alignment}(\mathbf{s}_{i-1}, \mathbf{h}_j) = \mathbf{v}^T \text{TanH}(\mathbf{W}_{es} \mathbf{s}_{i-1} + \mathbf{W}_{eh} \mathbf{h}_j),$$

First work

Bahdanau, Cho, and Bengio [4] introduced in 2014 the **first attention model** (“**alignment model**”)

Most **neural machine translation (NMT)** models follow an **encoder-decoder** approach. **RNNSearch** by Bahdanau, Cho, and Bengio [4]: the **encoder** is a **bidirectional GRU**,

$$\{\vec{\mathbf{h}}_j\}_{j=1}^{T_{\text{in}}} = \overrightarrow{\text{GRU}}(\{\mathbf{x}_j\}_{j=1}^{T_{\text{in}}}), \quad \{\overleftarrow{\mathbf{h}}_j\}_{j=1}^{T_{\text{in}}} = \overleftarrow{\text{GRU}}(\{\mathbf{x}_j\}_{j=1}^{T_{\text{in}}}), \quad \mathbf{h}_j = \begin{pmatrix} \vec{\mathbf{h}}_j \\ \overleftarrow{\mathbf{h}}_j \end{pmatrix}$$

the **decoder** is based on the “**context vector**” $\mathbf{c}_i$

$$\mathbf{c}_i = \sum_{j=1}^{T_{\text{in}}} a_{i,j} \mathbf{h}_j, \quad a_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k=1}^{T_{\text{in}}} \exp(e_{i,k})},$$

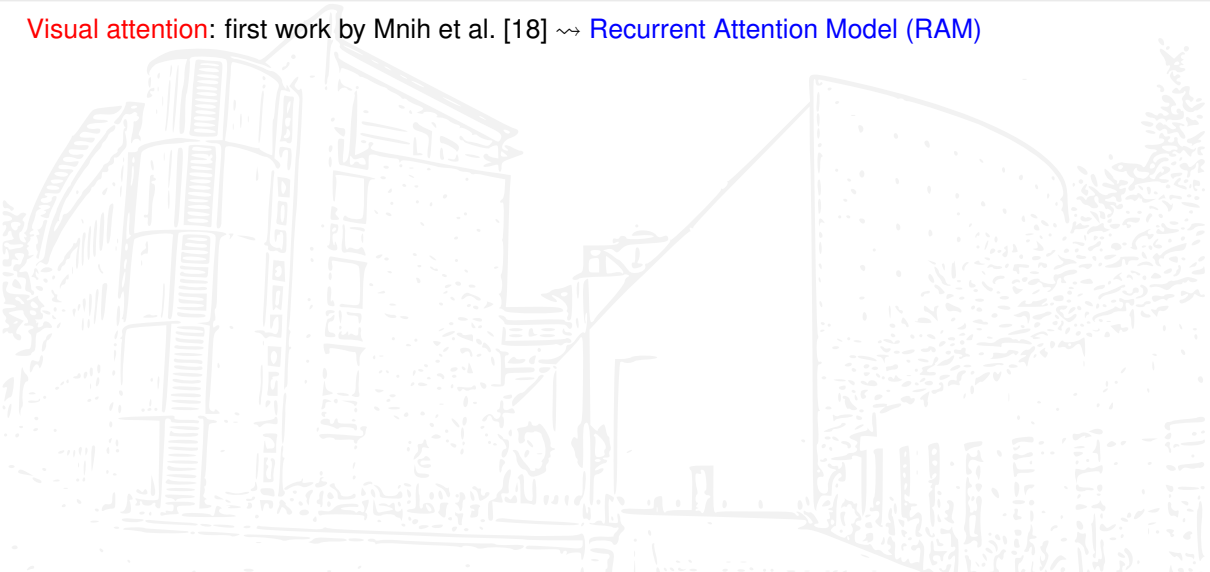
$$e_{i,j} = \text{alignment}(\mathbf{s}_{i-1}, \mathbf{h}_j) = \mathbf{v}^T \text{TanH}(\mathbf{W}_{es} \mathbf{s}_{i-1} + \mathbf{W}_{eh} \mathbf{h}_j),$$

where the **output probability** $\mathbf{p}_i$ for **output** $\mathbf{y}_i$ is modeled by an **RNN** f using **hidden state** $\mathbf{s}$,

$$\mathbf{p}_i = g(\mathbf{y}_{i-1}, \mathbf{s}_i, \mathbf{c}_i), \quad \mathbf{s}_i = f(\mathbf{s}_{i-1}, \mathbf{y}_{i-1}, \mathbf{c}_i).$$

CNN: soft and hard attention

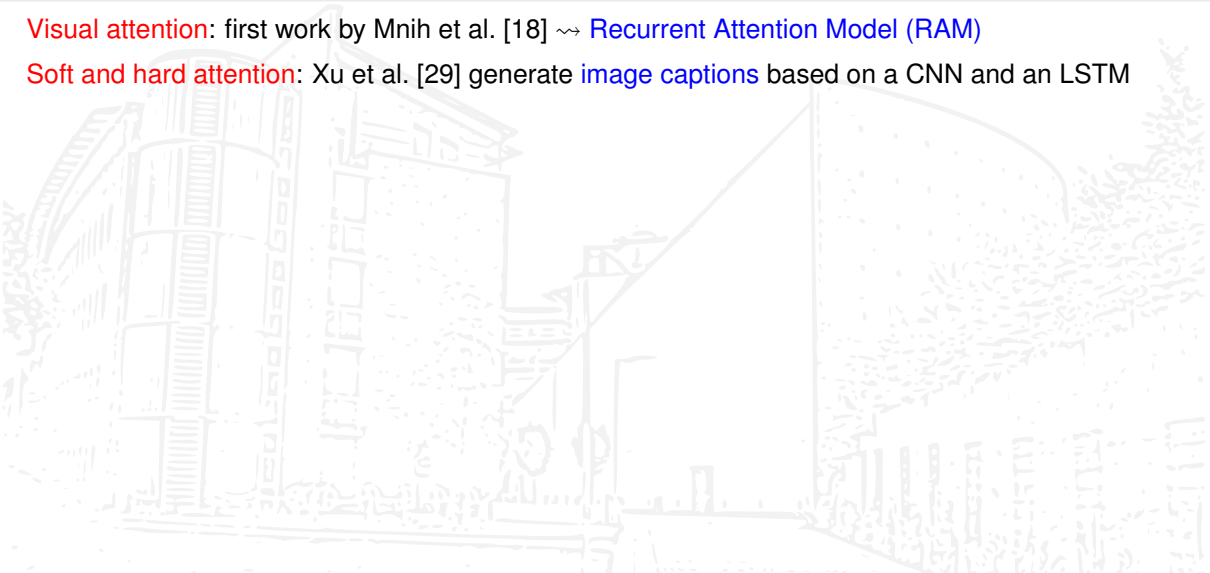
Visual attention: first work by Mnih et al. [18] $\rightsquigarrow$ **Recurrent Attention Model (RAM)**



CNN: soft and hard attention

Visual attention: first work by Mnih et al. [18] $\rightsquigarrow$ **Recurrent Attention Model (RAM)**

Soft and hard attention: Xu et al. [29] generate **image captions** based on a CNN and an LSTM



CNN: soft and hard attention

Visual attention: first work by Mnih et al. [18] $\rightsquigarrow$ **Recurrent Attention Model (RAM)**

Soft and hard attention: Xu et al. [29] generate **image captions** based on a CNN and an LSTM

hard attention: $\{0, 1\}$ -clipping to region of interest, **stochastic**, **training is hard**

soft attention: smooth $(0, 1)$ -“clipping”, **differentiable**, **training by backpropagation**

CNN: soft and hard attention

Visual attention: first work by Mnih et al. [18] $\rightsquigarrow$ **Recurrent Attention Model (RAM)**

Soft and hard attention: Xu et al. [29] generate **image captions** based on a CNN and an LSTM

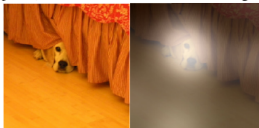
hard attention: $\{0, 1\}$ -clipping to region of interest, **stochastic**, **training is hard**

soft attention: smooth $(0, 1)$ -“clipping”, **differentiable**, **training by backpropagation**

Figure 4. Examples of attending to the correct object (*white* indicates the attended regions, *underlines* indicated the corresponding word)



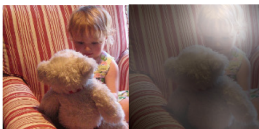
A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.

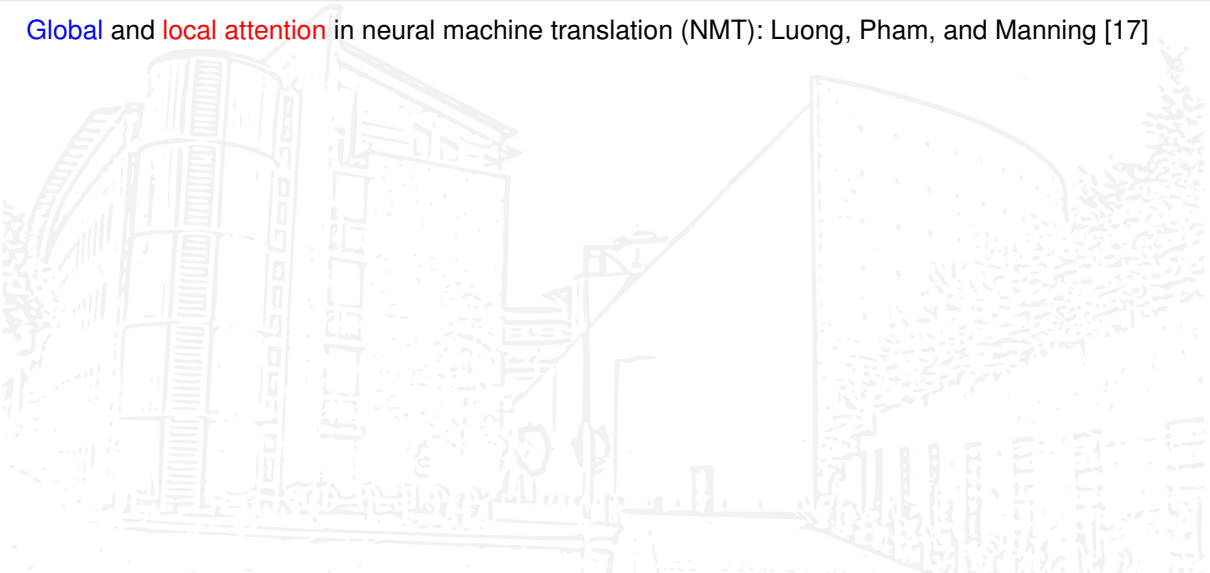


A giraffe standing in a forest with trees in the background.

Figure taken from Xu et al. [29].

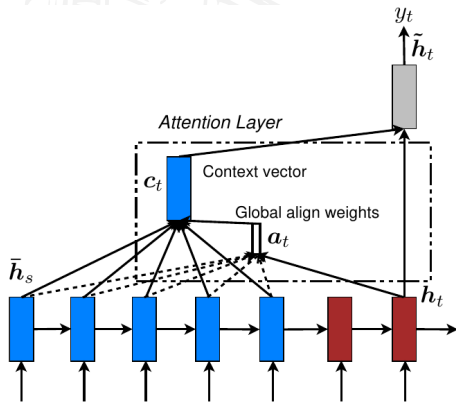
NMT: global and local attention

Global and local attention in neural machine translation (NMT): Luong, Pham, and Manning [17]



NMT: global and local attention

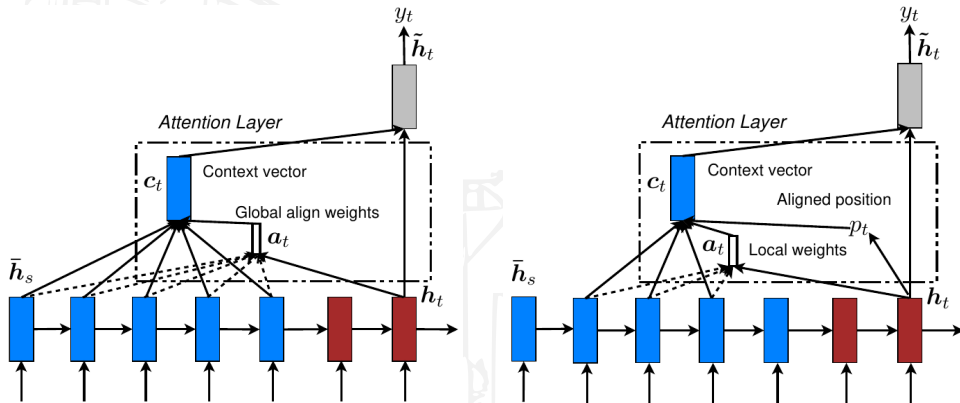
Global and local attention in neural machine translation (NMT): Luong, Pham, and Manning [17]



Figures taken from Luong, Pham, and Manning [17]. Left: global attention like in Bahdanau, Cho, and Bengio [4]

NMT: global and local attention

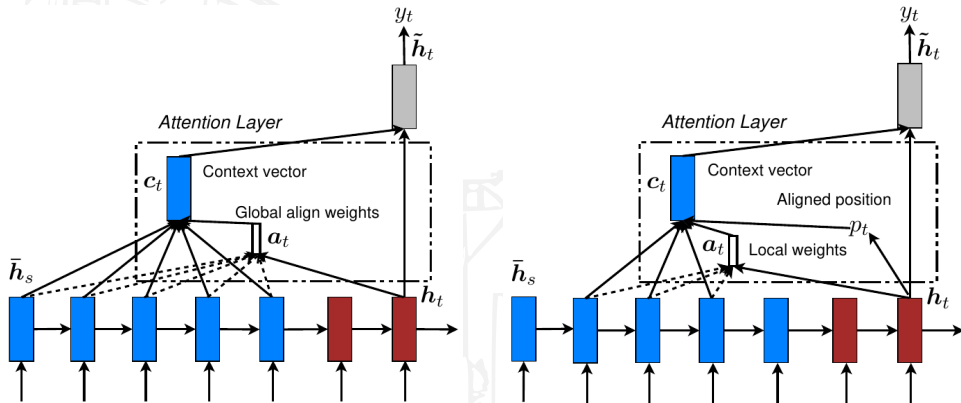
Global and local attention in neural machine translation (NMT): Luong, Pham, and Manning [17]



Figures taken from Luong, Pham, and Manning [17]. Left: global attention like in Bahdanau, Cho, and Bengio [4], right: local attention.

NMT: global and local attention

Global and local attention in neural machine translation (NMT): Luong, Pham, and Manning [17]



Figures taken from Luong, Pham, and Manning [17]. Left: global attention like in Bahdanau, Cho, and Bengio [4], right: local attention.

In contrast to hard attention: local attention differentiable almost everywhere $\rightsquigarrow$ backpropagation

Attention variants

Attention in Bahdanau, Cho, and Bengio [4] + **retrieval system terminology**:

$$\mathbf{c} = \mathbf{a}^T \mathbf{H}, \quad a_{i,j} = p(\mathbf{s}_{i-1}, \mathbf{h}_j) \quad \rightsquigarrow \quad \text{attention}(\mathbf{q}, \mathbf{K}, \mathbf{V}) = \sum_j \text{similarity}(\mathbf{q}, \mathbf{k}_j) \cdot \mathbf{v}_j,$$

where **q** “query” (decoder), **k** “key” (encoder), **v** “value” (encoder)

Attention variants

Attention in Bahdanau, Cho, and Bengio [4] + **retrieval system terminology**:

$$\mathbf{c} = \mathbf{a}^T \mathbf{H}, \quad a_{i,j} = p(\mathbf{s}_{i-1}, \mathbf{h}_j) \quad \rightsquigarrow \quad \text{attention}(\mathbf{q}, \mathbf{K}, \mathbf{V}) = \sum_j \text{similarity}(\mathbf{q}, \mathbf{k}_j) \cdot \mathbf{v}_j,$$

where **q** “query” (decoder), **k** “key” (encoder), **v** “value” (encoder)

The similarity is computed either **additive** or **multiplicative**, followed by a **SoftMax-layer**:

$$\text{add}(\mathbf{q}, \mathbf{k}) := \mathbf{v}^T a(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}),$$

$$\text{add}(\mathbf{q}, \mathbf{k}) := \ell(\mathbf{q}),$$

$$\text{mul}(\mathbf{q}, \mathbf{k}) := \frac{\mathbf{q}^T \mathbf{k}}{\|\mathbf{q}\| \|\mathbf{k}\|}, \quad \text{mul}(\mathbf{q}, \mathbf{k}) := \mathbf{q}^T \mathbf{k}, \quad \text{mul}(\mathbf{q}, \mathbf{k}) := \mathbf{q}^T \mathbf{W} \mathbf{k}, \quad \text{mul}(\mathbf{q}, \mathbf{k}) := \frac{\mathbf{q}^T \mathbf{k}}{\sqrt{d}}.$$

The additive first two types are used in Bahdanau, Cho, and Bengio [4] (also referred to as “concat”) and in Luong, Pham, and Manning [17] (positional, for local attention).

Attention variants

Attention in Bahdanau, Cho, and Bengio [4] + **retrieval system terminology**:

$$\mathbf{c} = \mathbf{a}^T \mathbf{H}, \quad a_{i,j} = p(\mathbf{s}_{i-1}, \mathbf{h}_j) \quad \rightsquigarrow \quad \text{attention}(\mathbf{q}, \mathbf{K}, \mathbf{V}) = \sum_j \text{similarity}(\mathbf{q}, \mathbf{k}_j) \cdot \mathbf{v}_j,$$

where **q** “query” (decoder), **k** “key” (encoder), **v** “value” (encoder)

The similarity is computed either **additive** or **multiplicative**, followed by a **SoftMax-layer**:

$$\text{add}(\mathbf{q}, \mathbf{k}) := \mathbf{v}^T a(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}),$$

$$\text{add}(\mathbf{q}, \mathbf{k}) := \ell(\mathbf{q}),$$

$$\text{mul}(\mathbf{q}, \mathbf{k}) := \frac{\mathbf{q}^T \mathbf{k}}{\|\mathbf{q}\| \|\mathbf{k}\|},$$

$$\text{mul}(\mathbf{q}, \mathbf{k}) := \mathbf{q}^T \mathbf{k},$$

$$\text{mul}(\mathbf{q}, \mathbf{k}) := \mathbf{q}^T \mathbf{W} \mathbf{k},$$

$$\text{mul}(\mathbf{q}, \mathbf{k}) := \frac{\mathbf{q}^T \mathbf{k}}{\sqrt{d}}.$$

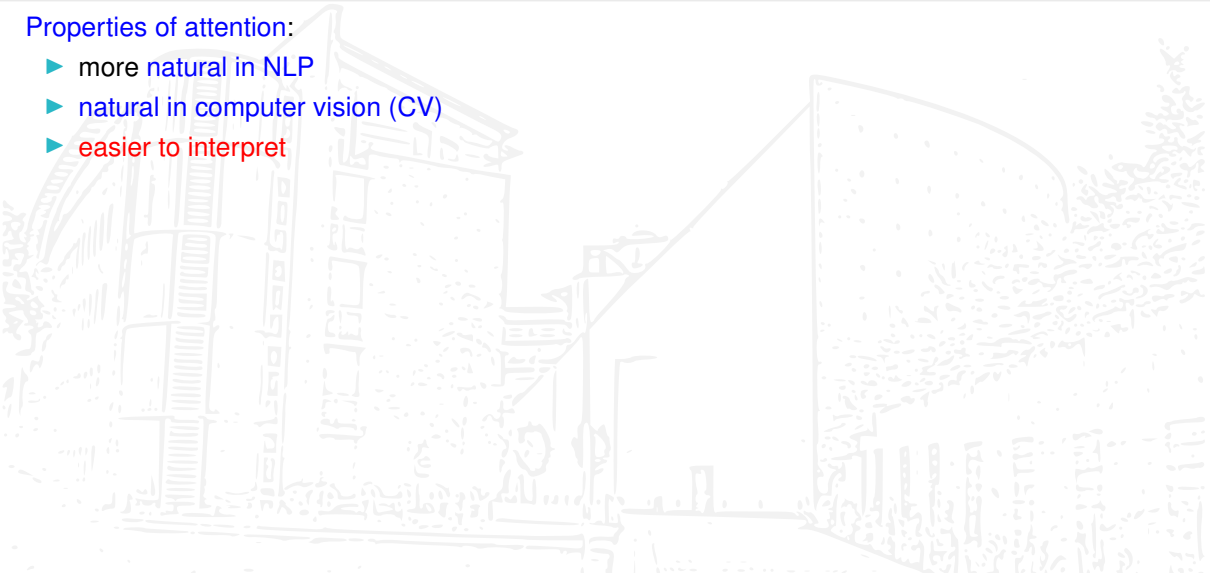
The additive first two types are used in Bahdanau, Cho, and Bengio [4] (also referred to as “concat”) and in Luong, Pham, and Manning [17] (positional, for local attention).

The last **multiplicative type** is used in the **transformer** (our second topic), Vaswani et al. [27].

Properties, other notable works, a timeline

Properties of attention:

- ▶ more natural in NLP
- ▶ natural in computer vision (CV)
- ▶ easier to interpret



Properties, other notable works, a timeline

Properties of attention:

- ▶ more natural in NLP
- ▶ natural in computer vision (CV)
- ▶ easier to interpret

Notable other works:

- ▶ **Recurrent Attention Model (RAM)**: Mnih et al. [18]
- ▶ **Memory Networks**: Weston, Chopra, and Bordes [28]

Properties, other notable works, a timeline

Properties of attention:

- ▶ more natural in NLP
- ▶ natural in computer vision (CV)
- ▶ easier to interpret

Notable other works:

- ▶ **Recurrent Attention Model (RAM)**: Mnih et al. [18]
- ▶ **Memory Networks**: Weston, Chopra, and Bordes [28]

Surveys on attention: Chaudhari et al. [8], Santana Correia and Colombini [25]

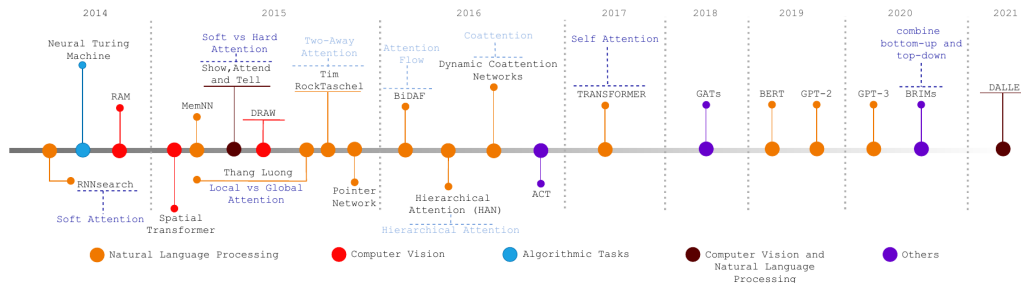
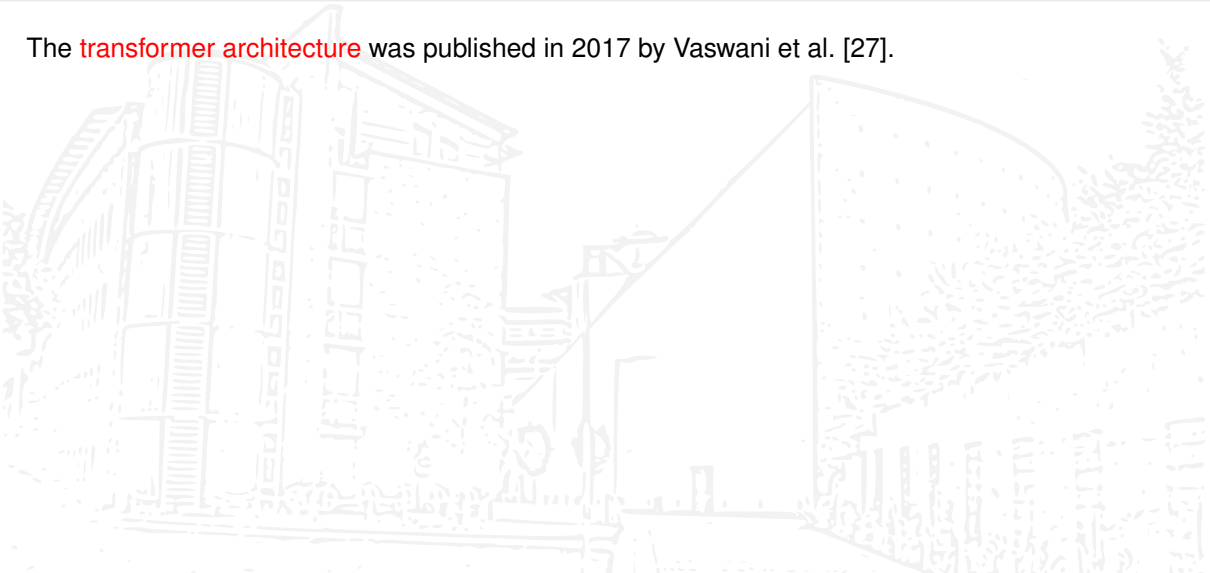


Figure taken from Santana Correia and Colombini [25].

Motivation

The **transformer architecture** was published in 2017 by Vaswani et al. [27].



Motivation

The **transformer architecture** was published in 2017 by Vaswani et al. [27].

RNN vs. Transformer networks:

RNN: e.g., LSTM or GRU

- ▶ problems w/ **long-term dependencies**
- ▶ **vanishing/exploding gradients**
- ▶ recurrence means **no parallel training**

Motivation

The **transformer architecture** was published in 2017 by Vaswani et al. [27].

RNN vs. Transformer networks:

RNN: e.g., LSTM or GRU

- ▶ problems w/ **long-term dependencies**
- ▶ **vanishing/exploding gradients**
- ▶ recurrence means **no parallel training**

Transformer: (next topic)

- ▶ **positional encoding** against long-term dependencies
- ▶ **no vanishing/exploding gradients**
- ▶ **parallel training possible**
- ▶ **scalability**, computational efficiency

Motivation

The **transformer architecture** was published in 2017 by Vaswani et al. [27].

RNN vs. Transformer networks:

RNN: e.g., LSTM or GRU

- ▶ problems w/ **long-term dependencies**
- ▶ **vanishing/exploding gradients**
- ▶ recurrence means **no parallel training**

Transformer: (next topic)

- ▶ **positional encoding** against long-term dependencies
- ▶ **no vanishing/exploding gradients**
- ▶ **parallel training possible**
- ▶ **scalability**, computational efficiency

Most networks in use nowadays are **based on the transformer**, e.g., **BERT** [10] and **GPT-4** [19].

Transformer

- The Transformer consists of an **encoder** (left) and a **decoder** (right), **no RNN** are involved

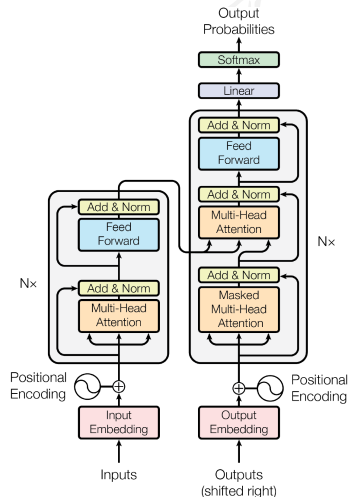
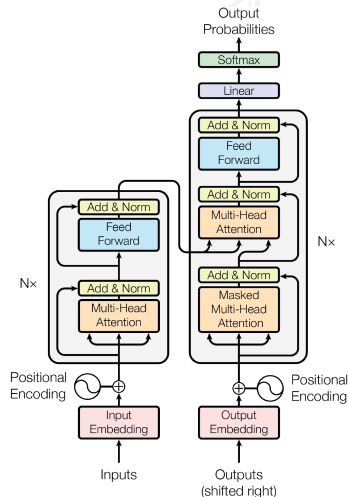


Figure 1: The Transformer - model architecture.

Figure taken from Vaswani et al. [27].

Transformer



- ▶ The Transformer consists of an **encoder** (left) and a **decoder** (right), **no RNN** are involved
- ▶ Inputs/shifted outputs are **embedded**, a **positional encoding** is added

Figure 1: The Transformer - model architecture.

Figure taken from Vaswani et al. [27].

Transformer

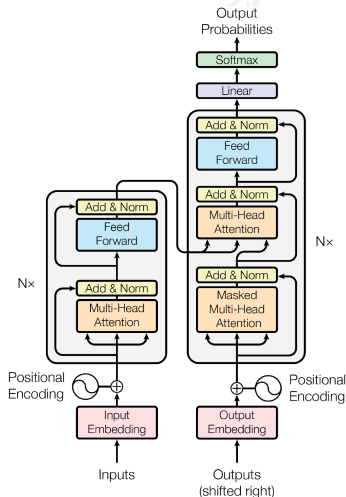


Figure 1: The Transformer - model architecture.

Figure taken from Vaswani et al. [27].

- ▶ The Transformer consists of an **encoder** (left) and a **decoder** (right), **no RNN** are involved
- ▶ Inputs/shifted outputs are **embedded**, a **positional encoding** is added
- ▶ Special types of attention, called **multi-head** and **masked multi-head attention**, are used in encoder and decoder $\rightsquigarrow$ “attention is all you need”

Transformer

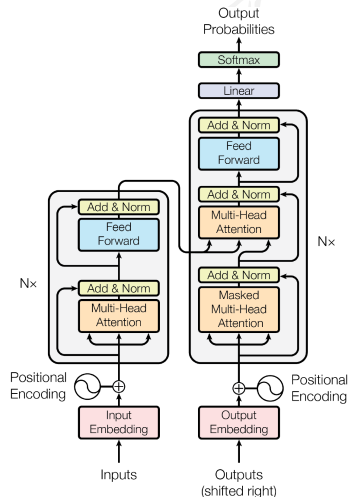


Figure 1: The Transformer - model architecture.

Figure taken from Vaswani et al. [27].

- ▶ The Transformer consists of an **encoder** (left) and a **decoder** (right), **no RNN** are involved
- ▶ Inputs/shifted outputs are **embedded**, a **positional encoding** is added
- ▶ Special types of attention, called **multi-head** and **masked multi-head attention**, are used in encoder and decoder $\rightsquigarrow$ “attention is all you need”
- ▶ **Skip-connections** and **layer normalization** [3] are used throughout $\rightsquigarrow$ similar to ResNet

Transformer

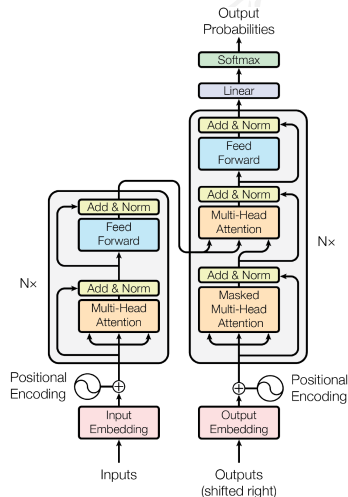


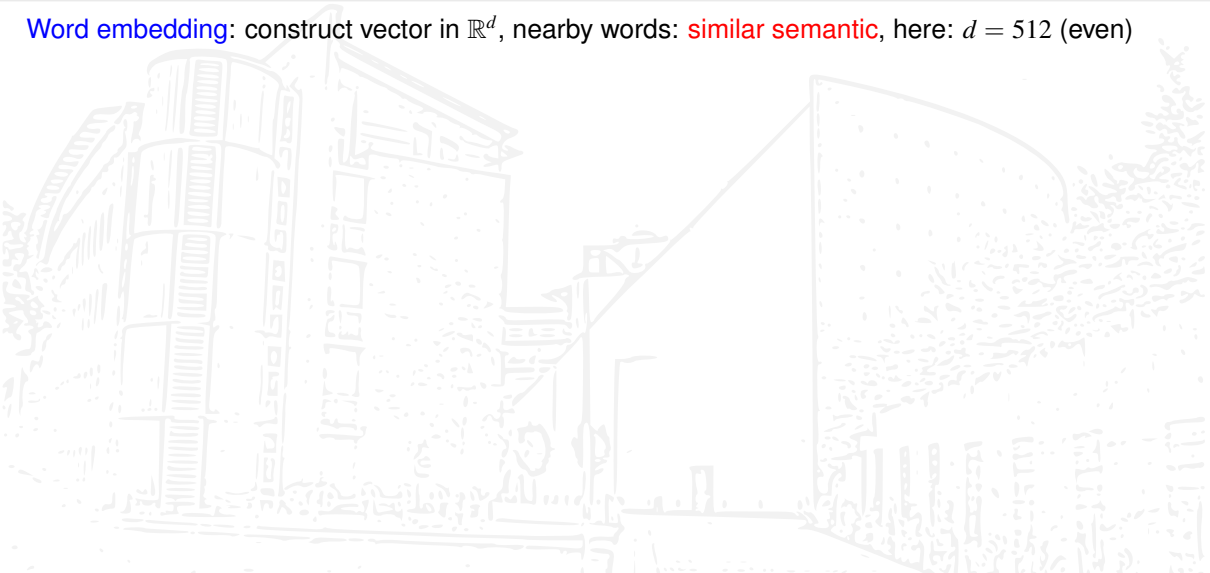
Figure 1: The Transformer - model architecture.

Figure taken from Vaswani et al. [27].

- ▶ The Transformer consists of an **encoder** (left) and a **decoder** (right), **no RNN** are involved
- ▶ Inputs/shifted outputs are **embedded**, a **positional encoding** is added
- ▶ Special types of attention, called **multi-head** and **masked multi-head attention**, are used in encoder and decoder $\rightsquigarrow$ “attention is all you need”
- ▶ **Skip-connections** and **layer normalization** [3] are used throughout $\rightsquigarrow$ similar to ResNet
- ▶ Encoder and decoder blocks are used **repeatedly** (denoted by $N \times$, $N = 6$ in the base model)

Word embedding and positional encoding

Word embedding: construct vector in $\mathbb{R}^d$, nearby words: **similar semantic**, here: $d = 512$ (even)



Word embedding and positional encoding

Word embedding: construct vector in $\mathbb{R}^d$, nearby words: **similar semantic**, here: $d = 512$ (even)

Positional encoding (Gehring et al. [12]): in [27] k th position encoded by $\mathbf{p}(k) \in \mathbb{R}^d$, where

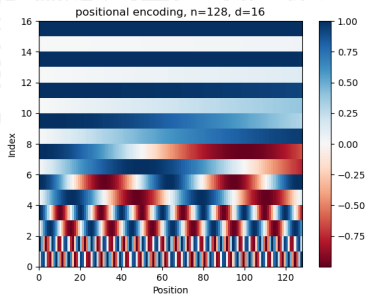
$$p_{2i}(k) = \sin\left(\frac{k}{10000^{2i/d}}\right), \quad p_{2i+1}(k) = \cos\left(\frac{k}{10000^{2i/d}}\right), \quad \|\mathbf{p}(k)\|_2 = \sqrt{\frac{d}{2}}, \quad 1 \leq k \leq n.$$

Word embedding and positional encoding

Word embedding: construct vector in $\mathbb{R}^d$, nearby words: **similar semantic**, here: $d = 512$ (even)

Positional encoding (Gehring et al. [12]): in [27] k th position encoded by $\mathbf{p}(k) \in \mathbb{R}^d$, where

$$p_{2i}(k) = \sin\left(\frac{k}{10000^{2i/d}}\right), \quad p_{2i+1}(k) = \cos\left(\frac{k}{10000^{2i/d}}\right), \quad \|\mathbf{p}(k)\|_2 = \sqrt{\frac{d}{2}}, \quad 1 \leq k \leq n.$$

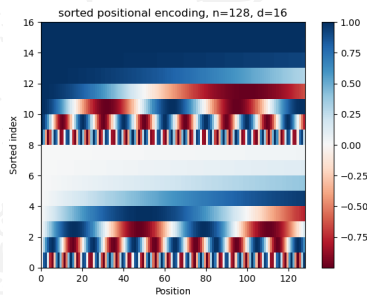
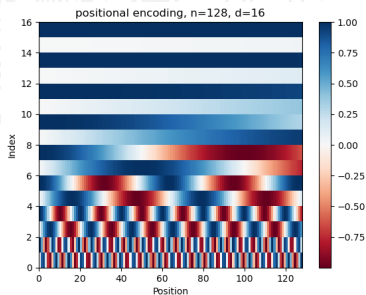


Word embedding and positional encoding

Word embedding: construct vector in $\mathbb{R}^d$, nearby words: **similar semantic**, here: $d = 512$ (even)

Positional encoding (Gehring et al. [12]): in [27] k th position encoded by $\mathbf{p}(k) \in \mathbb{R}^d$, where

$$p_{2i}(k) = \sin\left(\frac{k}{10000^{2i/d}}\right), \quad p_{2i+1}(k) = \cos\left(\frac{k}{10000^{2i/d}}\right), \quad \|\mathbf{p}(k)\|_2 = \sqrt{\frac{d}{2}}, \quad 1 \leq k \leq n.$$

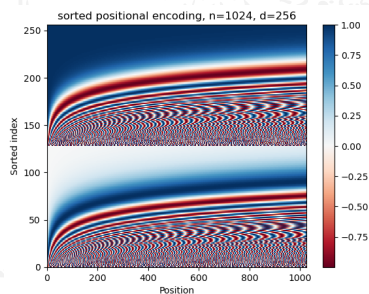
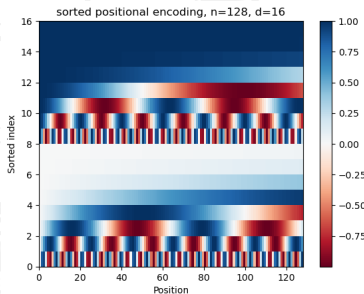
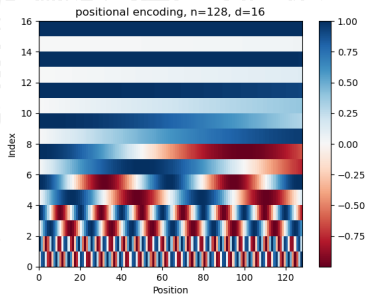


Word embedding and positional encoding

Word embedding: construct vector in $\mathbb{R}^d$, nearby words: **similar semantic**, here: $d = 512$ (even)

Positional encoding (Gehring et al. [12]): in [27] k th position encoded by $\mathbf{p}(k) \in \mathbb{R}^d$, where

$$p_{2i}(k) = \sin\left(\frac{k}{10000^{2i/d}}\right), \quad p_{2i+1}(k) = \cos\left(\frac{k}{10000^{2i/d}}\right), \quad \|\mathbf{p}(k)\|_2 = \sqrt{\frac{d}{2}}, \quad 1 \leq k \leq n.$$



Scaled dot-product attention

Transformer: utilizes **scaled dot-product (self-)attention**.

Scaled Dot-Product Attention

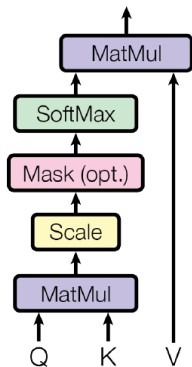


Figure taken from Vaswani et al. [27].

Scaled dot-product attention

Scaled Dot-Product Attention

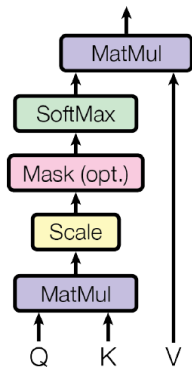


Figure taken from Vaswani et al. [27].

Transformer: utilizes **scaled dot-product (self-)attention**.

Suppose that for **one input** sequence we have queries $Q \in \mathbb{R}^{s_q \times d_k}$, keys $K \in \mathbb{R}^{s_k \times d_k}$, and values $V \in \mathbb{R}^{s_k \times d_v}$.

Scaled dot-product attention

Scaled Dot-Product Attention

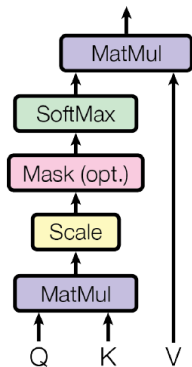


Figure taken from Vaswani et al. [27].

Transformer: utilizes **scaled dot-product (self-)attention**.

Suppose that for **one input** sequence we have queries $\mathbf{Q} \in \mathbb{R}^{s_q \times d_k}$, keys $\mathbf{K} \in \mathbb{R}^{s_k \times d_k}$, and values $\mathbf{V} \in \mathbb{R}^{s_k \times d_v}$.

The **scaled dot-product attention** is computed as

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{SoftMax} \left(\mathbf{M} + \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V} \in \mathbb{R}^{s_q \times d_v},$$

w/ **mask** $\mathbf{M} \in \mathbb{R}^{s_q \times s_k}$ and SoftMax is applied **rowwise**.

Scaled dot-product attention

Scaled Dot-Product Attention

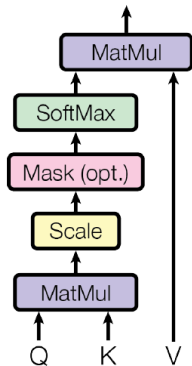


Figure taken from Vaswani et al. [27].

Transformer: utilizes **scaled dot-product (self-)attention**.

Suppose that for **one input** sequence we have queries $\mathbf{Q} \in \mathbb{R}^{s_q \times d_k}$, keys $\mathbf{K} \in \mathbb{R}^{s_k \times d_k}$, and values $\mathbf{V} \in \mathbb{R}^{s_k \times d_v}$.

The **scaled dot-product attention** is computed as

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{SoftMax} \left(\mathbf{M} + \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V} \in \mathbb{R}^{s_q \times d_v},$$

w/ **mask** $\mathbf{M} \in \mathbb{R}^{s_q \times s_k}$ and SoftMax is applied **rowwise**.

Two kinds of masks: **padding mask**, **look-ahead mask** (only in masked multi-head attention $\rightsquigarrow$ next slide). Entries: 0, $-\infty$.

Scaled dot-product attention

Scaled Dot-Product Attention

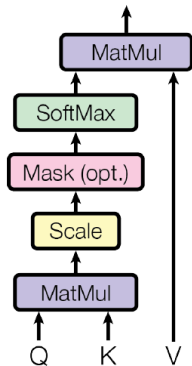


Figure taken from Vaswani et al. [27].

Transformer: utilizes **scaled dot-product (self-)attention**.

Suppose that for **one input** sequence we have queries $\mathbf{Q} \in \mathbb{R}^{s_q \times d_k}$, keys $\mathbf{K} \in \mathbb{R}^{s_k \times d_k}$, and values $\mathbf{V} \in \mathbb{R}^{s_k \times d_v}$.

The **scaled dot-product attention** is computed as

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{SoftMax} \left(\mathbf{M} + \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V} \in \mathbb{R}^{s_q \times d_v},$$

w/ **mask** $\mathbf{M} \in \mathbb{R}^{s_q \times s_k}$ and SoftMax is applied **rowwise**.

Two kinds of masks: **padding mask**, **look-ahead mask** (only in masked multi-head attention $\rightsquigarrow$ next slide). Entries: 0, $-\infty$.

Later used w/ **b batches** and **h heads** $\rightsquigarrow$ **4-tensors**

$$\mathbf{Q} \in \mathbb{R}^{b \times h \times s_q \times d_k}, \mathbf{K} \in \mathbb{R}^{b \times h \times s_k \times d_k}, \mathbf{V} \in \mathbb{R}^{b \times h \times s_k \times d_v}$$

Multi-head attention

Multi-head attention is based on h scaled dot-product attention layers, in the paper $h = 8$.

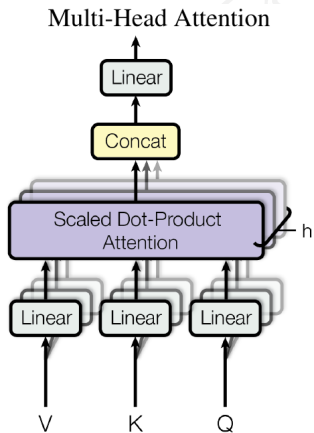


Figure taken from Vaswani et al. [27].

Multi-head attention

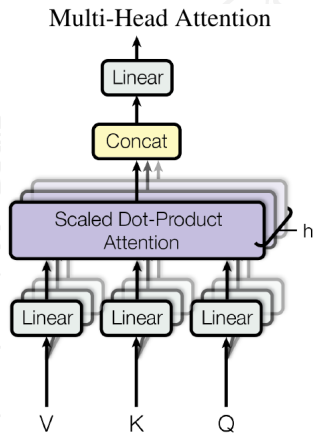


Figure taken from Vaswani et al. [27].

Multi-head attention is based on h **scaled dot-product attention layers**, in the paper $h = 8$.

Incoming V, K, Q are processed **linearly** and handed to the **attention layers** (here denoted w/o bias),

$$\text{Head}_i = \text{Attention}(\mathbf{QW}_i^Q, \mathbf{KW}_i^K, \mathbf{VW}_i^V),$$

$$\mathbf{W}_i^Q \in \mathbb{R}^{d \times d_k}, \mathbf{W}_i^K \in \mathbb{R}^{d \times d_k}, \mathbf{W}_i^V \in \mathbb{R}^{d \times d_v}, d_k = d_v = d/h = 64.$$

Multi-head attention

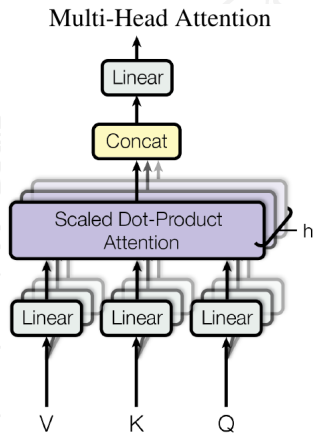


Figure taken from Vaswani et al. [27].

Multi-head attention is based on h **scaled dot-product attention layers**, in the paper $h = 8$.

Incoming V, K, Q are processed **linearly** and handed to the **attention layers** (here denoted w/o bias),

$$\text{Head}_i = \text{Attention}(\mathbf{QW}_i^Q, \mathbf{KW}_i^K, \mathbf{VW}_i^V),$$

$$\mathbf{W}_i^Q \in \mathbb{R}^{d \times d_k}, \mathbf{W}_i^K \in \mathbb{R}^{d \times d_k}, \mathbf{W}_i^V \in \mathbb{R}^{d \times d_v}, d_k = d_v = d/h = 64.$$

The output is **concatenated** and processed **linearly**,

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{concat}(\text{Head}_1, \dots, \text{Head}_h)\mathbf{W},$$

where $\mathbf{W} \in \mathbb{R}^{hd_v \times d}$. (Again we omitted the bias.)

Multi-head attention

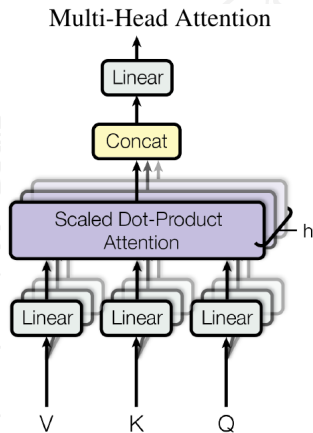


Figure taken from Vaswani et al. [27].

Multi-head attention is based on h **scaled dot-product attention layers**, in the paper $h = 8$.

Incoming V, K, Q are processed **linearly** and handed to the **attention layers** (here denoted w/o bias),

$$\text{Head}_i = \text{Attention}(\mathbf{QW}_i^Q, \mathbf{KW}_i^K, \mathbf{VW}_i^V),$$

$$\mathbf{W}_i^Q \in \mathbb{R}^{d \times d_k}, \mathbf{W}_i^K \in \mathbb{R}^{d \times d_k}, \mathbf{W}_i^V \in \mathbb{R}^{d \times d_v}, d_k = d_v = d/h = 64.$$

The output is **concatenated** and processed **linearly**,

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{concat}(\text{Head}_1, \dots, \text{Head}_h) \mathbf{W},$$

where $\mathbf{W} \in \mathbb{R}^{hd_v \times d}$. (Again we omitted the bias.)

Encoder: $\mathbf{Q} = \mathbf{K} = \mathbf{V}$, **decoder:** $\mathbf{V} = \mathbf{K} \rightsquigarrow$ next slide

Other layers / attention types / which mask to use?

Feed Forward: one hidden layer w/ ReLU, 2048 neurons, everything else (after embedding) d -dimensional, $d = 512$

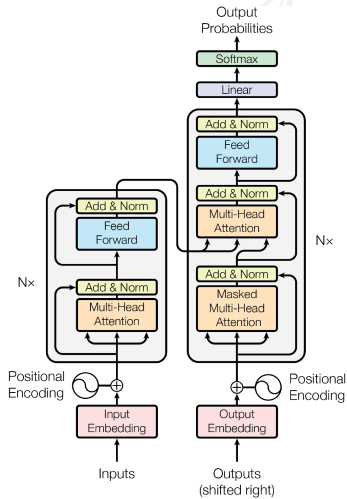


Figure 1: The Transformer - model architecture.

Figure taken from Vaswani et al. [27].

Other layers / attention types / which mask to use?

Feed Forward: one hidden layer w/ ReLU, 2048 neurons, everything else (after embedding) d -dimensional, $d = 512$

Encoder: self-attention, $Q = K = V$

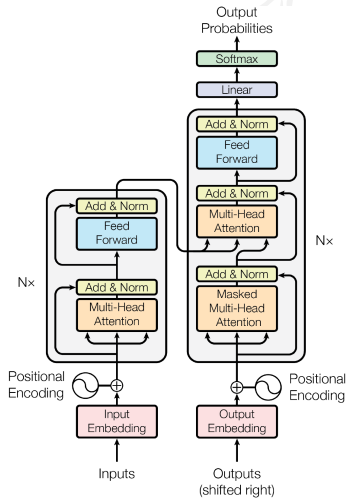


Figure 1: The Transformer - model architecture.

Figure taken from Vaswani et al. [27].

Other layers / attention types / which mask to use?

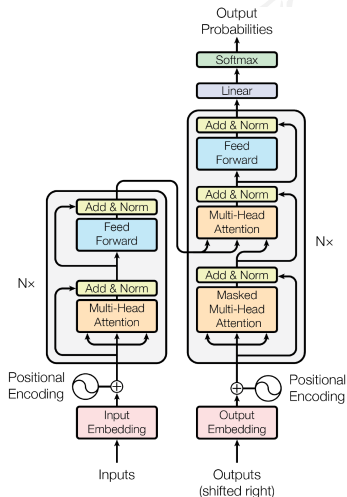


Figure 1: The Transformer - model architecture.

Figure taken from Vaswani et al. [27].

Feed Forward: one hidden layer w/ ReLU, 2048 neurons, everything else (after embedding) d -dimensional, $d = 512$

Encoder: self-attention, $Q = K = V$

Decoder: first self-attention, then attention to output of encoder

Other layers / attention types / which mask to use?

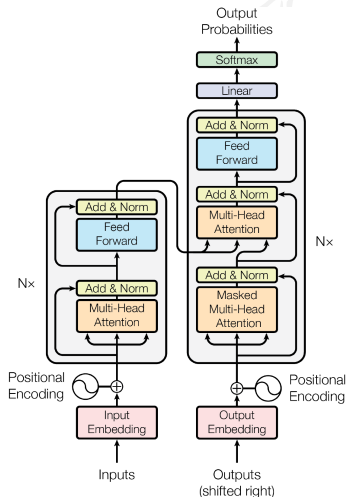


Figure 1: The Transformer - model architecture.

Figure taken from Vaswani et al. [27].

Feed Forward: one hidden layer w/ ReLU, 2048 neurons, everything else (after embedding) d -dimensional, $d = 512$

Encoder: self-attention, $\mathbf{Q} = \mathbf{K} = \mathbf{V}$

Decoder: first self-attention, then attention to output of encoder

Decoder self-attention: $\mathbf{Q} = \mathbf{K} = \mathbf{V}$, no future words, thus mask! Matrix must be lower triangular ($-\infty$ in upper part):

$$\mathbf{M} = \begin{pmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{pmatrix} \rightsquigarrow \begin{pmatrix} 1.0 & 0.0 & 0.0 \\ 0.2 & 0.8 & 0.0 \\ 0.2 & 0.1 & 0.7 \end{pmatrix}$$

Other layers / attention types / which mask to use?

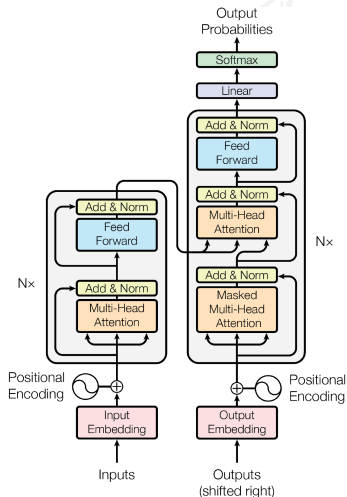


Figure 1: The Transformer - model architecture.

Figure taken from Vaswani et al. [27].

Feed Forward: one hidden layer w/ ReLU, 2048 neurons, everything else (after embedding) d -dimensional, $d = 512$

Encoder: self-attention, $\mathbf{Q} = \mathbf{K} = \mathbf{V}$

Decoder: first self-attention, then attention to output of encoder

Decoder self-attention: $\mathbf{Q} = \mathbf{K} = \mathbf{V}$, no future words, thus mask! Matrix must be lower triangular ($-\infty$ in upper part):

$$\mathbf{M} = \begin{pmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{pmatrix} \rightsquigarrow \begin{pmatrix} 1.0 & 0.0 & 0.0 \\ 0.2 & 0.8 & 0.0 \\ 0.2 & 0.1 & 0.7 \end{pmatrix}$$

Encoder-decoder attention: $\mathbf{K} = \mathbf{V}$ final output of encoder, $\mathbf{Q}$ from decoder self-attention

Other layers / attention types / which mask to use?

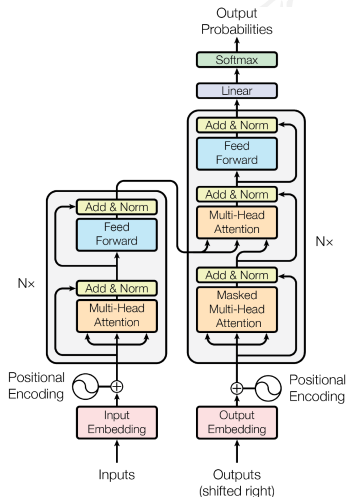


Figure 1: The Transformer - model architecture.

Figure taken from Vaswani et al. [27].

Feed Forward: one hidden layer w/ ReLU, 2048 neurons, everything else (after embedding) d -dimensional, $d = 512$

Encoder: self-attention, $\mathbf{Q} = \mathbf{K} = \mathbf{V}$

Decoder: first self-attention, then attention to output of encoder

Decoder self-attention: $\mathbf{Q} = \mathbf{K} = \mathbf{V}$, no future words, thus mask! Matrix must be lower triangular ($-\infty$ in upper part):

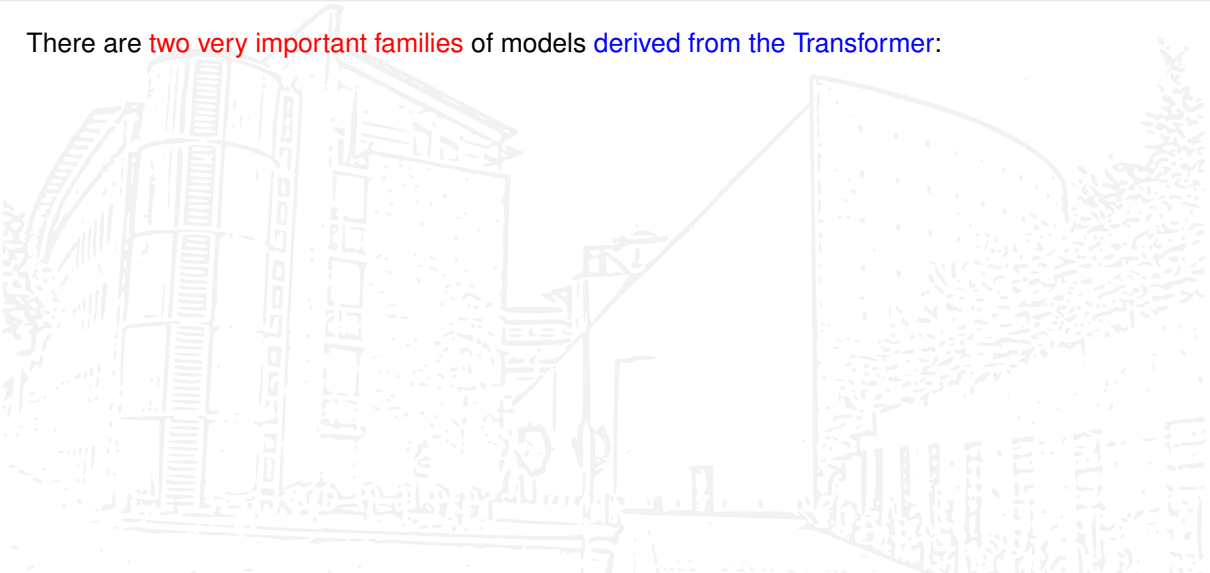
$$\mathbf{M} = \begin{pmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{pmatrix} \rightsquigarrow \begin{pmatrix} 1.0 & 0.0 & 0.0 \\ 0.2 & 0.8 & 0.0 \\ 0.2 & 0.1 & 0.7 \end{pmatrix}$$

Encoder-decoder attention: $\mathbf{K} = \mathbf{V}$ final output of encoder, $\mathbf{Q}$ from decoder self-attention

Output probabilities: linear layer + SoftMax

BERT & GPT

There are **two very important families** of models **derived from the Transformer**:



BERT & GPT

There are **two very important families** of models **derived from the Transformer**:

Stack encoders: This idea was pursued by researchers at **Google** and results in **Bidirectional Encoder Representations from Transformers (BERT)**, see Devlin et al. [10], new models are **RoBERTa**, **DistilBERT**, **ALBERT**, ...

BERT & GPT

There are **two very important families** of models **derived from the Transformer**:

Stack encoders: This idea was pursued by researchers at **Google** and results in **Bidirectional Encoder Representations from Transformers (BERT)**, see Devlin et al. [10], new models are **RoBERTa**, **DistilBERT**, **ALBERT**, ...

Stack decoders: This idea was pursued by researchers at **OpenAI** and results in **Generative Pre-trained Transformer (GPT)**:

GPT: 2018, Radford et al. [20]

GPT-2: 2019, Radford et al. [21]

GPT-3: 2020, Brown et al. [7]

GPT-3.5: 2022

GPT-4: 2023, OpenAI [19]

BERT & GPT

There are **two very important families** of models **derived from the Transformer**:

Stack encoders: This idea was pursued by researchers at **Google** and results in **Bidirectional Encoder Representations from Transformers (BERT)**, see Devlin et al. [10], new models are **RoBERTa**, **DistilBERT**, **ALBERT**, ...

Stack decoders: This idea was pursued by researchers at **OpenAI** and results in **Generative Pre-trained Transformer (GPT)**:

GPT: 2018, Radford et al. [20]

GPT-2: 2019, Radford et al. [21]

GPT-3: 2020, Brown et al. [7]

GPT-3.5: 2022

GPT-4: 2023, OpenAI [19]

Large database of **transformer models**: <https://huggingface.co/docs/transformers>.

BERT

Model (in **Feed Forward** always $h = 4H$ hidden neurons):

model	layers (L)	dimension (H)	heads (A)	parameters	download link
base	12	768	12	110M	BERT base
large	24	1024	16	340M	BERT large

Table: BERT models

BERT

Model (in **Feed Forward** always $h = 4H$ hidden neurons):

model	layers (L)	dimension (H)	heads (A)	parameters	download link
base	12	768	12	110M	BERT base
large	24	1024	16	340M	BERT large

Table: BERT models

The **training of BERT** is split into two parts:

pre-train: to understand language, based on

- ▶ **Masked Language Model (MLM)**
- ▶ **Next Sentence Prediction (NSP)**

BERT

Model (in **Feed Forward** always $h = 4H$ hidden neurons):

model	layers (L)	dimension (H)	heads (A)	parameters	download link
base	12	768	12	110M	BERT base
large	24	1024	16	340M	BERT large

Table: BERT models

The **training of BERT** is split into two parts:

pre-train: to understand language, based on

- ▶ **Masked Language Model (MLM)**
- ▶ **Next Sentence Prediction (NSP)**

fine-tune: depending on the application, **change input and output layers** $\rightsquigarrow$ **machine translation, sentiment analysis, construct abstract of text, answer questions** (wished I'd had one of those in school)

Parameter count of BERT

How many parameters has BERT base ($L = 12$, $d = H = 768$, $h = A = 12$)?

layer	2-dim	1-dim	sum	value
MHA	$3h \cdot d \times d/h + d \times d = 4d^2$	$3h \cdot d/h + d = 4d$	$4d^2 + 4d$	2.362.368
FFN	$d \times 4d + 4d \times d = 8d^2$	$4d + d = 5d$	$8d^2 + 5d$	4.722.432
LN	0	$2d$	$2d$	1.536
EncLayer	MHA + LN + FFN + LN		$12d^2 + 13d$	7.087.872
Encoder	$L \cdot \text{EncLayer}$		$L(12d^2 + 13d)$	85.054.464
Tok. Emb.	$t \times d = td$	0	td	23.440.896
Pos. Emb.	$p \times d = pd$	0	pd	393.216
Seg. Emb.	$2 \times d = 2d$	0	$2d$	1.536
Embedding	Tok. + Pos. + Seg. + LN		$(t + p + 4)d$	23.837.184
Output	$d \times d = d^2$	d	$d^2 + d$	590.592
BERT	Embedding + Encoder + Output			109.482.240

Table: Parameter count for BERT

This version of BERT has $t = 30522$ tokens, $p = 512$ positions and uses two sentences/segments

Pre-training of BERT

During **pre-training**, BERT acts **autoencoder-like** simultaneously on the two tasks:

Masked Language Model: alter 15% of the words in the sentences, 80% masked by [MASK], 10% replaced by random word, 10% correct word.

Next Sentence Prediction: choose sentences A and B. 50% of the time B directly follows A, labeled as `IsNext`, 50% it is a random sentence from the corpus, labeled as `NotNext`.

Pre-training of BERT

During **pre-training**, BERT acts **autoencoder-like** simultaneously on the two tasks:

Masked Language Model: alter 15% of the words in the sentences, 80% masked by [MASK], 10% replaced by random word, 10% correct word.

Next Sentence Prediction: choose sentences A and B. 50% of the time B directly follows A, labeled as **IsNext**, 50% it is a random sentence from the corpus, labeled as **NotNext**.

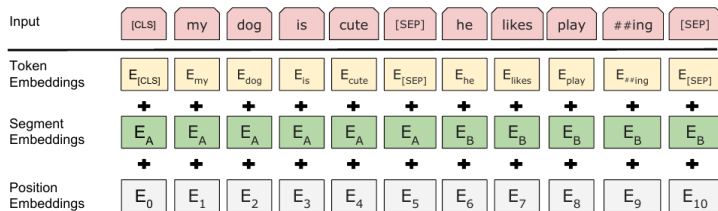


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

Figure taken from Devlin et al. [10].

Overall training of BERT

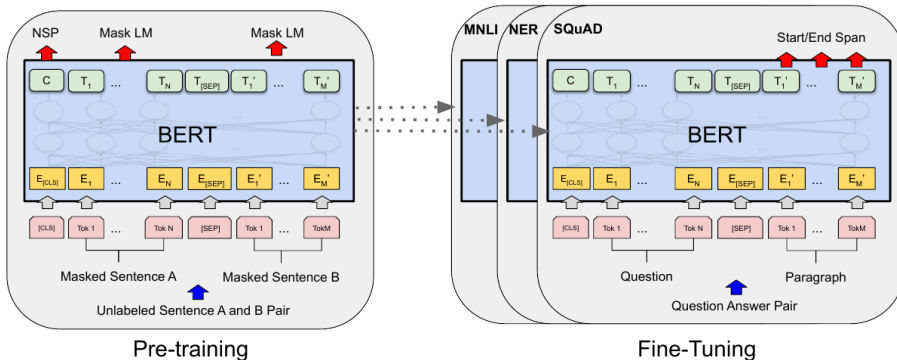
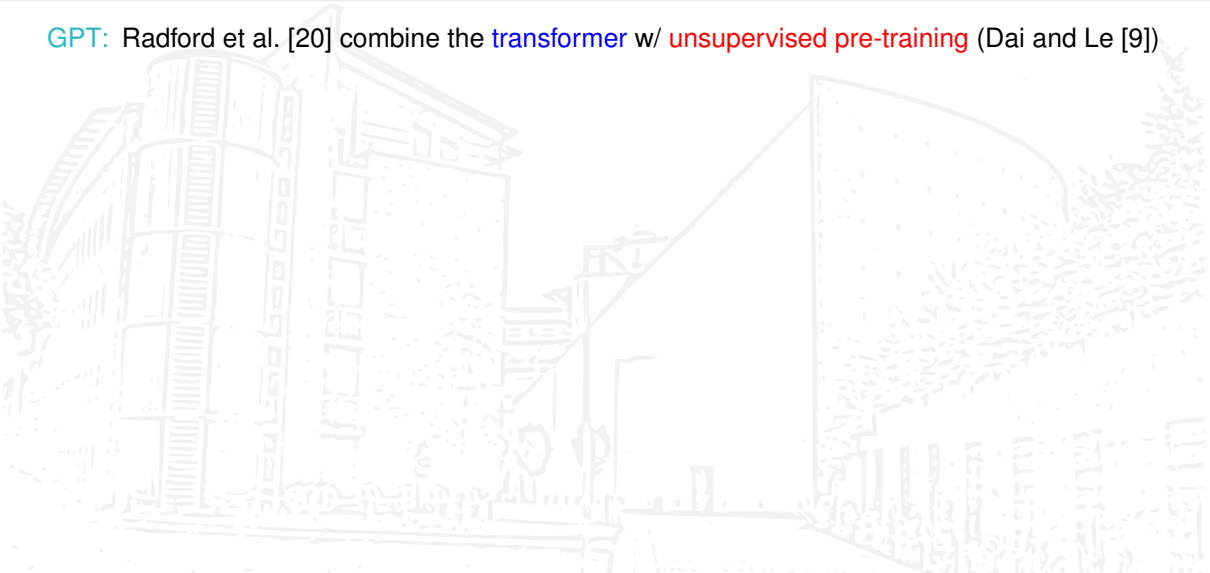


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

Figure taken from Devlin et al. [10].

GPT, GPT-2, GPT-3, GPT-4

GPT: Radford et al. [20] combine the **transformer** w/ **unsupervised pre-training** (Dai and Le [9])



GPT, GPT-2, GPT-3, GPT-4

GPT: Radford et al. [20] combine the **transformer** w/ **unsupervised pre-training** (Dai and Le [9])

GPT-2: Radford et al. [21] used a larger version of GPT w/ **1.5 billion parameters**, trained on a dataset of **8 million web pages**, 10 times number of parameters and training data than GPT

GPT, GPT-2, GPT-3, GPT-4

- GPT:** Radford et al. [20] combine the **transformer** w/ **unsupervised pre-training** (Dai and Le [9])
- GPT-2:** Radford et al. [21] used a larger version of GPT w/ **1.5 billion parameters**, trained on a dataset of **8 million web pages**, 10 times number of parameters and training data than GPT
- GPT-3:** Brown et al. [7] used a model w/ **175 billion parameters** and used **45TB of compressed plaintext** (before filtering) / **570GB** (after filtering) as training data

GPT, GPT-2, GPT-3, GPT-4

- GPT:** Radford et al. [20] combine the **transformer** w/ **unsupervised pre-training** (Dai and Le [9])
- GPT-2:** Radford et al. [21] used a larger version of GPT w/ **1.5 billion parameters**, trained on a dataset of **8 million web pages**, 10 times number of parameters and training data than GPT
- GPT-3:** Brown et al. [7] used a model w/ **175 billion parameters** and used **45TB of compressed plaintext** (before filtering) / **570GB** (after filtering) as training data
- GPT-4:** "...this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar." [19, p. 2]
- GPT-x is trained like our minimal character predicting RNN $\rightsquigarrow$ **autoregressive model**

GPT, GPT-2, GPT-3, GPT-4

- GPT:** Radford et al. [20] combine the **transformer** w/ **unsupervised pre-training** (Dai and Le [9])
- GPT-2:** Radford et al. [21] used a larger version of GPT w/ **1.5 billion parameters**, trained on a dataset of **8 million web pages**, 10 times number of parameters and training data than GPT
- GPT-3:** Brown et al. [7] used a model w/ **175 billion parameters** and used **45TB of compressed plaintext** (before filtering) / **570GB** (after filtering) as training data
- GPT-4:** "...this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar." [19, p. 2]

GPT-x is trained like our minimal character predicting RNN $\rightsquigarrow$ **autoregressive model**

GPT-x is a **Zero-Shot** or **Few-Shot Learner**: once pre-trained, only very few steps of fine-tuning are necessary (w/ only a **few training examples**)

GPT, GPT-2, GPT-3, GPT-4

- GPT:** Radford et al. [20] combine the **transformer** w/ **unsupervised pre-training** (Dai and Le [9])
- GPT-2:** Radford et al. [21] used a larger version of GPT w/ **1.5 billion parameters**, trained on a dataset of **8 million web pages**, 10 times number of parameters and training data than GPT
- GPT-3:** Brown et al. [7] used a model w/ **175 billion parameters** and used **45TB of compressed plaintext** (before filtering) / **570GB** (after filtering) as training data
- GPT-4:** "...this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar." [19, p. 2]

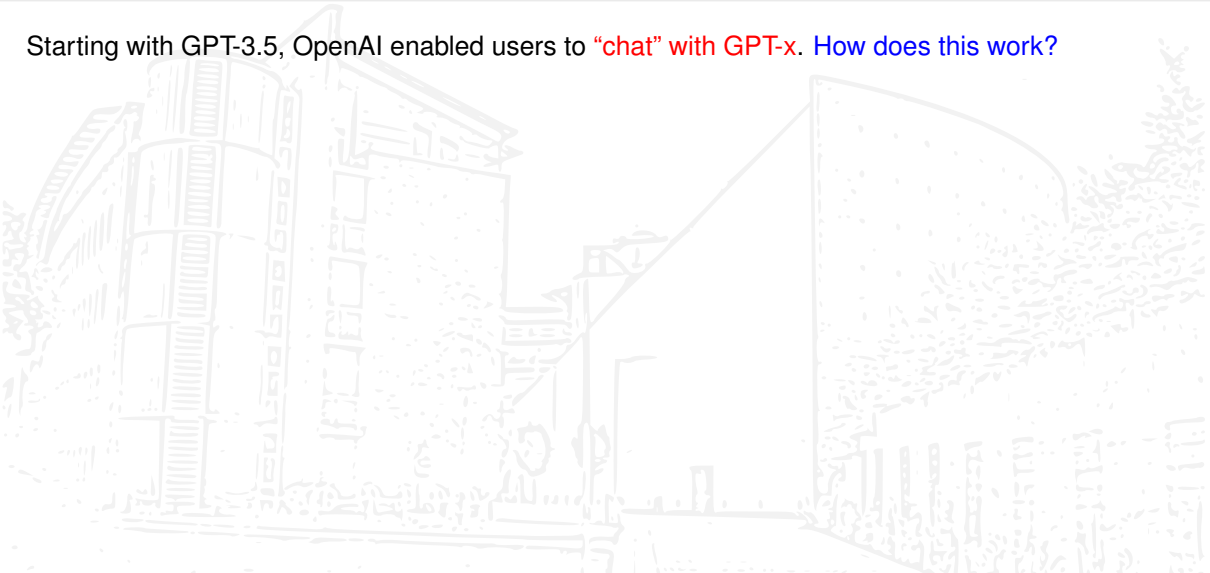
GPT-x is trained like our minimal character predicting RNN $\rightsquigarrow$ **autoregressive model**

GPT-x is a **Zero-Shot** or **Few-Shot Learner**: once pre-trained, only very few steps of fine-tuning are necessary (w/ only a **few training examples**)

Trained GPT-2 versions were **withheld** to prevent **malicious use**, GPT-3 can only be used via frontend, OpenAI **licensed** GPT-3 to **Microsoft**.

ChatGPT

Starting with GPT-3.5, OpenAI enabled users to “chat” with GPT-x. How does this work?



ChatGPT

Starting with GPT-3.5, OpenAI enabled users to “chat” with GPT-x. How does this work?

GPT is an autoregressive model that takes the tokens into account that have been feed in before.

ChatGPT

Starting with GPT-3.5, OpenAI enabled users to “chat” with GPT-x. How does this work?

GPT is an autoregressive model that takes the tokens into account that have been feed in before.

By “steering” the model into a certain realm of the latent space, the model acts as a chatbot. This is done via “prompting”, there is a hidden prompt that is feed into the model, only part of its output is visible to the users.

ChatGPT

Starting with GPT-3.5, OpenAI enabled users to “chat” with GPT-x. How does this work?

GPT is an autoregressive model that takes the tokens into account that have been feed in before.

By “steering” the model into a certain realm of the latent space, the model acts as a chatbot. This is done via “prompting”, there is a hidden prompt that is feed into the model, only part of its output is visible to the users.

To make this interactive, the autoregressive use is stopped at certain tokens, e.g., “User:”.

ChatGPT

Starting with GPT-3.5, OpenAI enabled users to “chat” with GPT-x. How does this work?

GPT is an autoregressive model that takes the tokens into account that have been feed in before.

By “steering” the model into a certain realm of the latent space, the model acts as a chatbot. This is done via “prompting”, there is a hidden prompt that is feed into the model, only part of its output is visible to the users.

To make this interactive, the autoregressive use is stopped at certain tokens, e.g., “User:”.

The training of ChatGPT proceeds in three steps:

ChatGPT

Starting with GPT-3.5, OpenAI enabled users to “chat” with GPT-x. How does this work?

GPT is an autoregressive model that takes the tokens into account that have been feed in before.

By “steering” the model into a certain realm of the latent space, the model acts as a chatbot. This is done via “prompting”, there is a hidden prompt that is feed into the model, only part of its output is visible to the users.

To make this interactive, the autoregressive use is stopped at certain tokens, e.g., “User:”.

The training of ChatGPT proceeds in three steps:

- ▶ Train the GPT model on some large corpus of text. This is known as pre-training and is self-supervised learning.

ChatGPT

Starting with GPT-3.5, OpenAI enabled users to “chat” with GPT-x. How does this work?

GPT is an autoregressive model that takes the tokens into account that have been feed in before.

By “steering” the model into a certain realm of the latent space, the model acts as a chatbot. This is done via “prompting”, there is a hidden prompt that is feed into the model, only part of its output is visible to the users.

To make this interactive, the autoregressive use is stopped at certain tokens, e.g., “User:”.

The training of ChatGPT proceeds in three steps:

- ▶ Train the GPT model on some large corpus of text. This is known as pre-training and is self-supervised learning.
- ▶ Train the model to reply to questions with answers, this is based on supervised learning.

ChatGPT

Starting with GPT-3.5, OpenAI enabled users to “chat” with GPT-x. How does this work?

GPT is an autoregressive model that takes the tokens into account that have been feed in before.

By “steering” the model into a certain realm of the latent space, the model acts as a chatbot. This is done via “prompting”, there is a hidden prompt that is feed into the model, only part of its output is visible to the users.

To make this interactive, the autoregressive use is stopped at certain tokens, e.g., “User:”.

The training of ChatGPT proceeds in three steps:

- ▶ Train the GPT model on some large corpus of text. This is known as pre-training and is self-supervised learning.
- ▶ Train the model to reply to questions with answers, this is based on supervised learning.
- ▶ Finetune the answers by using reinforcement learning from human feedback (RLHF).

“BERTology”

BERTology: analysis of BERT and GPT-x, relation to (classical) NLP



“BERTology”

BERTology: analysis of **BERT** and **GPT-x**, relation to (classical) NLP

Some **BERTology** papers are collected at

<https://huggingface.co/docs/transformers/bertology>

“BERTology”

BERTology: analysis of **BERT** and **GPT-x**, relation to (classical) NLP

Some **BERTology** papers are collected at

<https://huggingface.co/docs/transformers/bertology>

We mention **three interesting works** concerning the **analysis**:

- ▶ Tenney, Das, and Pavlick [26] consider **similarities** between **BERT** and **classical NLP**, the treatment in the layers of the encoder follow roughly “natural” patterns (like we saw in CNNs)

“BERTology”

BERTology: analysis of **BERT** and **GPT-x**, relation to (classical) NLP

Some **BERTology papers** are collected at

<https://huggingface.co/docs/transformers/bertology>

We mention **three interesting works** concerning the **analysis**:

- ▶ Tenney, Das, and Pavlick [26] consider **similarities** between **BERT** and **classical NLP**, the treatment in the layers of the encoder follow roughly “natural” patterns (like we saw in CNNs)
- ▶ Rogers, Kovaleva, and Rumshisky [23] published a “**primer in BERTology**” that contains pointers to many interesting works, e.g., [26]. **Environmental aspects** are also discussed in this survey: <https://www.youtube.com/watch?v=1aaWuAum5HY>

“BERTology”

BERTology: analysis of **BERT** and **GPT-x**, relation to (classical) NLP

Some **BERTology papers** are collected at

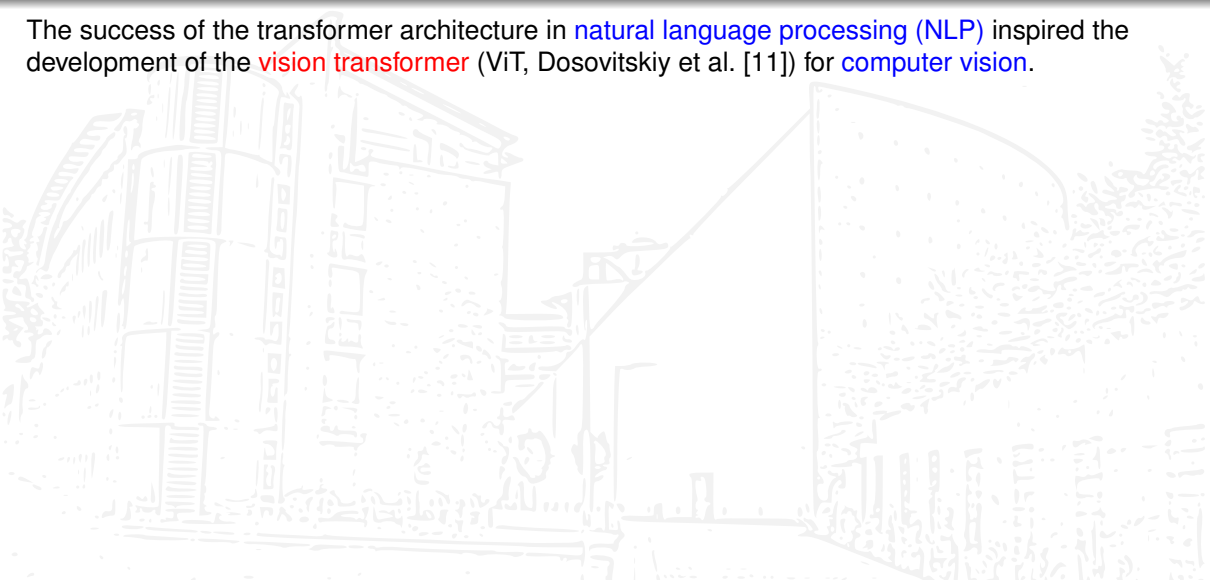
<https://huggingface.co/docs/transformers/bertology>

We mention **three interesting works** concerning the **analysis**:

- ▶ Tenney, Das, and Pavlick [26] consider **similarities** between **BERT** and **classical NLP**, the treatment in the layers of the encoder follow roughly “natural” patterns (like we saw in CNNs)
- ▶ Rogers, Kovaleva, and Rumshisky [23] published a “**primer in BERTology**” that contains pointers to many interesting works, e.g., [26]. **Environmental aspects** are also discussed in this survey: <https://www.youtube.com/watch?v=1aaWuAum5HY>
- ▶ Biderman and Raff [5] discuss how GPT-J (another GPT variant) generates completions to online homeworks that are **mostly plagiarizes**

Vision Transformer (ViT)

The success of the transformer architecture in **natural language processing (NLP)** inspired the development of the **vision transformer** (ViT, Dosovitskiy et al. [11]) for **computer vision**.



Vision Transformer (ViT)

The success of the transformer architecture in **natural language processing (NLP)** inspired the development of the **vision transformer (ViT, Dosovitskiy et al. [11])** for **computer vision**.

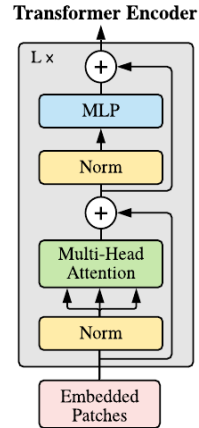
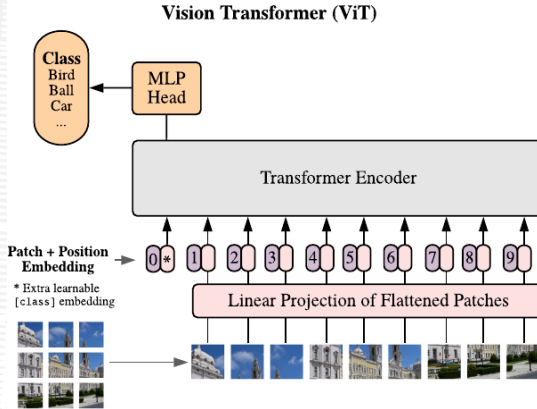
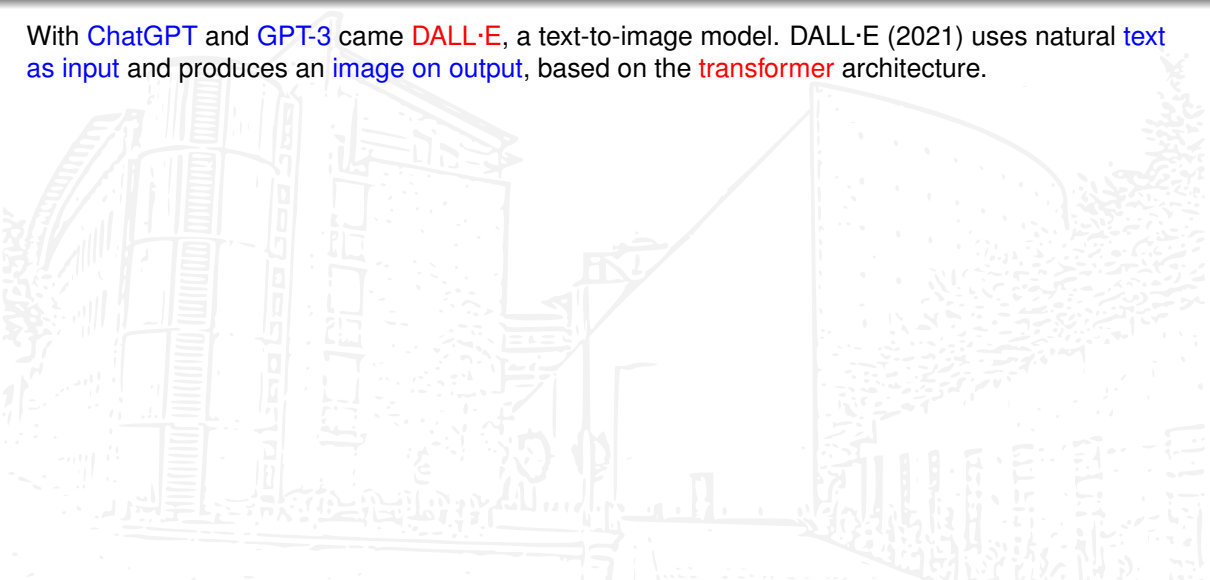


Image taken from Dosovitskiy et al. [11]

DALL·E & CLIP

With ChatGPT and GPT-3 came DALL·E, a text-to-image model. DALL·E (2021) uses natural text as input and produces an image on output, based on the transformer architecture.



DALL·E & CLIP

With **ChatGPT** and **GPT-3** came **DALL·E**, a text-to-image model. DALL·E (2021) uses natural **text as input** and produces an **image on output**, based on the **transformer** architecture.

DALL·E uses output of **contrastive language-image pre-training** (**CLIP**, Radford et al. [22]).

(1) Contrastive pre-training

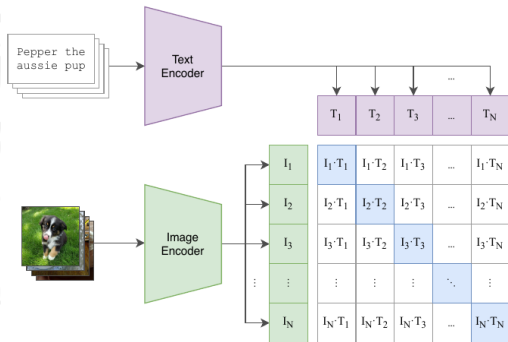
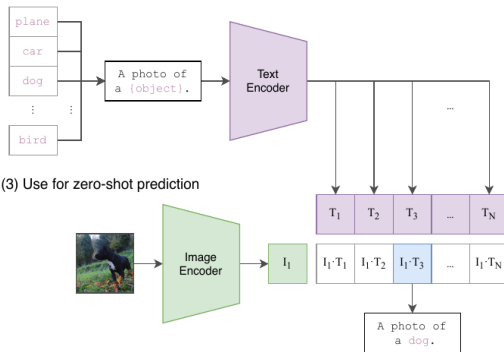
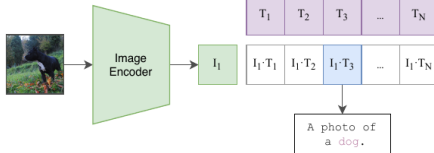


Image taken from Radford et al. [22]

(2) Create dataset classifier from label text

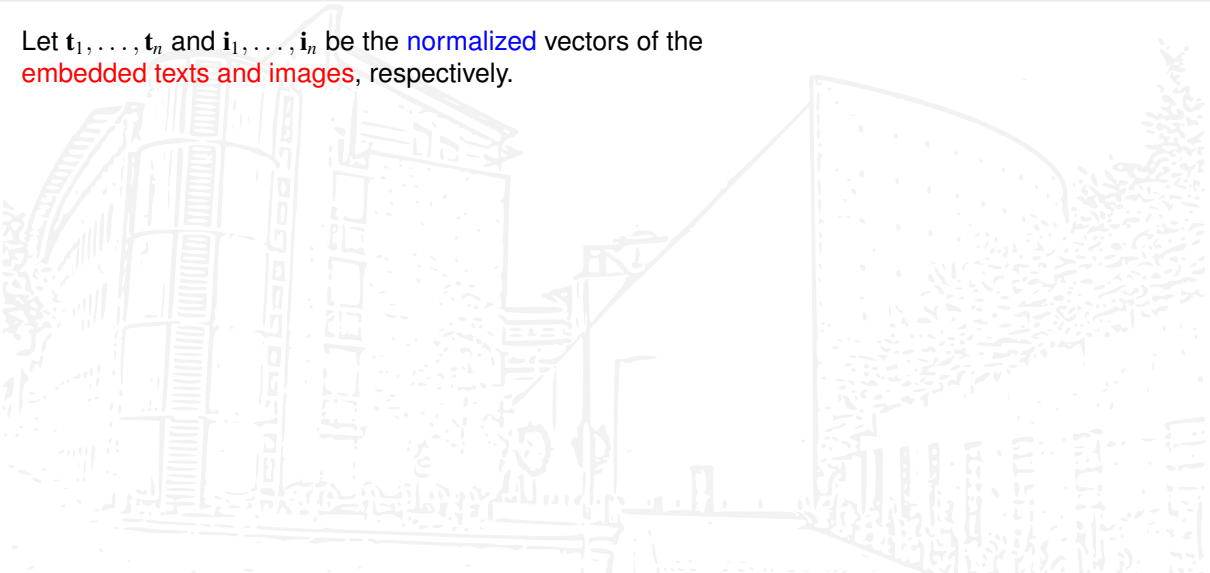


(3) Use for zero-shot prediction



Loss function used in CLIP

Let $\mathbf{t}_1, \dots, \mathbf{t}_n$ and $\mathbf{i}_1, \dots, \mathbf{i}_n$ be the **normalized** vectors of the **embedded texts and images**, respectively.



Loss function used in CLIP

Let $\mathbf{t}_1, \dots, \mathbf{t}_n$ and $\mathbf{i}_1, \dots, \mathbf{i}_n$ be the **normalized** vectors of the **embedded texts and images**, respectively. CLIP uses the **sum** of the **cross-entropy loss** on SoftMax of the **rows and columns** of the matrix

$$\mathbf{M}(\mathbf{i}, \mathbf{t}) := \begin{pmatrix} \langle \mathbf{i}_1, \mathbf{t}_1 \rangle & \cdots & \langle \mathbf{i}_1, \mathbf{t}_n \rangle \\ \vdots & \ddots & \vdots \\ \langle \mathbf{i}_n, \mathbf{t}_1 \rangle & \cdots & \langle \mathbf{i}_n, \mathbf{t}_n \rangle \end{pmatrix}$$

with respect to the **labels** $1, \dots, n$

Loss function used in CLIP

Let $\mathbf{t}_1, \dots, \mathbf{t}_n$ and $\mathbf{i}_1, \dots, \mathbf{i}_n$ be the **normalized** vectors of the **embedded texts and images**, respectively. CLIP uses the **sum** of the **cross-entropy loss** on SoftMax of the **rows and columns** of the matrix

$$\mathbf{M}(\mathbf{i}, \mathbf{t}) := \begin{pmatrix} \langle \mathbf{i}_1, \mathbf{t}_1 \rangle & \cdots & \langle \mathbf{i}_1, \mathbf{t}_n \rangle \\ \vdots & \ddots & \vdots \\ \langle \mathbf{i}_n, \mathbf{t}_1 \rangle & \cdots & \langle \mathbf{i}_n, \mathbf{t}_n \rangle \end{pmatrix}$$

with respect to the **labels** $1, \dots, n$

```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T) #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t)/2
```

Image taken from Radford et al. [22]

Loss function used in CLIP

Let $\mathbf{t}_1, \dots, \mathbf{t}_n$ and $\mathbf{i}_1, \dots, \mathbf{i}_n$ be the **normalized** vectors of the **embedded texts and images**, respectively. CLIP uses the **sum** of the **cross-entropy loss** on SoftMax of the **rows and columns** of the matrix

$$\mathbf{M}(\mathbf{i}, \mathbf{t}) := \begin{pmatrix} \langle \mathbf{i}_1, \mathbf{t}_1 \rangle & \cdots & \langle \mathbf{i}_1, \mathbf{t}_n \rangle \\ \vdots & \ddots & \vdots \\ \langle \mathbf{i}_n, \mathbf{t}_1 \rangle & \cdots & \langle \mathbf{i}_n, \mathbf{t}_n \rangle \end{pmatrix}$$

with respect to the **labels** $1, \dots, n$, i.e., up to a scalar,

$$-\sum_{i=1}^n \log \frac{\exp(\langle \mathbf{i}_i, \mathbf{t}_i \rangle)}{\sum_{j=1}^n \exp(\langle \mathbf{i}_i, \mathbf{t}_j \rangle)} - \sum_{j=1}^n \log \frac{\exp(\langle \mathbf{i}_j, \mathbf{t}_j \rangle)}{\sum_{i=1}^n \exp(\langle \mathbf{i}_i, \mathbf{t}_j \rangle)}.$$

```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T) #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t)/2

Image taken from Radford et al. [22]
```

Loss function used in CLIP

Let $\mathbf{t}_1, \dots, \mathbf{t}_n$ and $\mathbf{i}_1, \dots, \mathbf{i}_n$ be the **normalized** vectors of the **embedded texts and images**, respectively. CLIP uses the **sum** of the **cross-entropy loss** on SoftMax of the **rows and columns** of the matrix

$$\mathbf{M}(\mathbf{i}, \mathbf{t}) := \begin{pmatrix} \langle \mathbf{i}_1, \mathbf{t}_1 \rangle & \cdots & \langle \mathbf{i}_1, \mathbf{t}_n \rangle \\ \vdots & \ddots & \vdots \\ \langle \mathbf{i}_n, \mathbf{t}_1 \rangle & \cdots & \langle \mathbf{i}_n, \mathbf{t}_n \rangle \end{pmatrix}$$

with respect to the **labels** $1, \dots, n$, i.e., up to a scalar,

$$-\sum_{i=1}^n \log \frac{\exp(\langle \mathbf{i}_i, \mathbf{t}_i \rangle)}{\sum_{j=1}^n \exp(\langle \mathbf{i}_i, \mathbf{t}_j \rangle)} - \sum_{j=1}^n \log \frac{\exp(\langle \mathbf{i}_j, \mathbf{t}_j \rangle)}{\sum_{i=1}^n \exp(\langle \mathbf{i}_i, \mathbf{t}_j \rangle)}.$$

The **cosine similarity** should satisfy $\mathbf{M}(\mathbf{i}, \mathbf{t}) \approx \mathbf{I}$.

```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T) #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

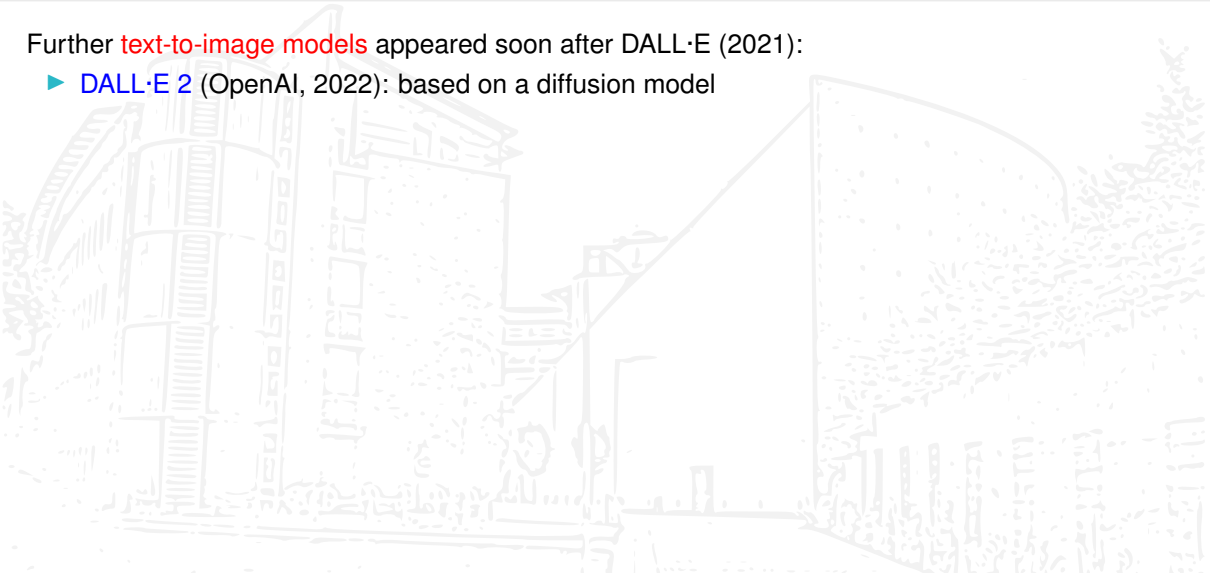
# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t)/2

Image taken from Radford et al. [22]
```

Multimodal models

Further **text-to-image models** appeared soon after DALL·E (2021):

- ▶ **DALL·E 2** (OpenAI, 2022): based on a diffusion model



Multimodal models

Further **text-to-image models** appeared soon after DALL·E (2021):

- ▶ **DALL·E 2** (OpenAI, 2022): based on a diffusion model
- ▶ **Imagen** (Google Brain, 2022): based on a diffusion model

Multimodal models

Further **text-to-image models** appeared soon after DALL·E (2021):

- ▶ **DALL·E 2** (OpenAI, 2022): based on a diffusion model
- ▶ **Imagen** (Google Brain, 2022): based on a diffusion model
- ▶ **Stable Diffusion** (Stability AI, 2022): based on a latent diffusion model

Multimodal models

Further **text-to-image models** appeared soon after DALL·E (2021):

- ▶ **DALL·E 2** (OpenAI, 2022): based on a diffusion model
- ▶ **Imagen** (Google Brain, 2022): based on a diffusion model
- ▶ **Stable Diffusion** (Stability AI, 2022): based on a latent diffusion model
- ▶ **Midjourney** (Midjourney, Inc., 2022): based on a diffusion model

Multimodal models

Further **text-to-image models** appeared soon after DALL·E (2021):

- ▶ **DALL·E 2** (OpenAI, 2022): based on a diffusion model
- ▶ **Imagen** (Google Brain, 2022): based on a diffusion model
- ▶ **Stable Diffusion** (Stability AI, 2022): based on a latent diffusion model
- ▶ **Midjourney** (Midjourney, Inc., 2022): based on a diffusion model
- ▶ **DALL·E 3** (OpenAI, 2023): based on a diffusion model

Multimodal models

Further **text-to-image models** appeared soon after DALL·E (2021):

- ▶ **DALL·E 2** (OpenAI, 2022): based on a diffusion model
- ▶ **Imagen** (Google Brain, 2022): based on a diffusion model
- ▶ **Stable Diffusion** (Stability AI, 2022): based on a latent diffusion model
- ▶ **Midjourney** (Midjourney, Inc., 2022): based on a diffusion model
- ▶ **DALL·E 3** (OpenAI, 2023): based on a diffusion model

Of all these text-to-image models, only **Stable Diffusion** is published with **code, weights, and training details**:

<https://huggingface.co/CompVis/stable-diffusion>

Multimodal models

Further **text-to-image models** appeared soon after DALL·E (2021):

- ▶ **DALL·E 2** (OpenAI, 2022): based on a diffusion model
- ▶ **Imagen** (Google Brain, 2022): based on a diffusion model
- ▶ **Stable Diffusion** (Stability AI, 2022): based on a latent diffusion model
- ▶ **Midjourney** (Midjourney, Inc., 2022): based on a diffusion model
- ▶ **DALL·E 3** (OpenAI, 2023): based on a diffusion model

Of all these text-to-image models, only **Stable Diffusion** is published with **code, weights, and training details**:

`https://huggingface.co/CompVis/stable-diffusion`

There are also more **general multimodal models**, we mention **Flamingo** (Alayrac et al. [1]) and **BLIP** (Li et al. [16]). Both work with **text and images** and can answer questions about these.

Multimodal models

Further **text-to-image models** appeared soon after DALL·E (2021):

- ▶ **DALL·E 2** (OpenAI, 2022): based on a **diffusion model**
- ▶ **Imagen** (Google Brain, 2022): based on a **diffusion model**
- ▶ **Stable Diffusion** (Stability AI, 2022): based on a **latent diffusion model**
- ▶ **Midjourney** (Midjourney, Inc., 2022): based on a **diffusion model**
- ▶ **DALL·E 3** (OpenAI, 2023): based on a **diffusion model**

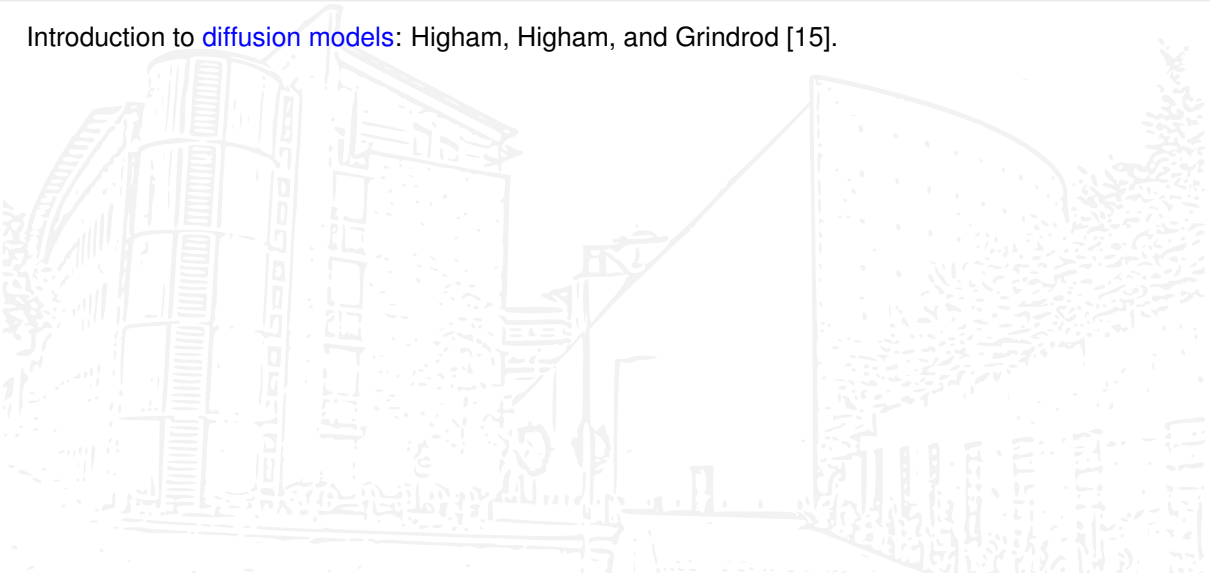
Of all these text-to-image models, only **Stable Diffusion** is published with **code, weights, and training details**:

`https://huggingface.co/CompVis/stable-diffusion`

There are also more **general multimodal models**, we mention **Flamingo** (Alayrac et al. [1]) and **BLIP** (Li et al. [16]). Both work with **text and images** and can answer questions about these.

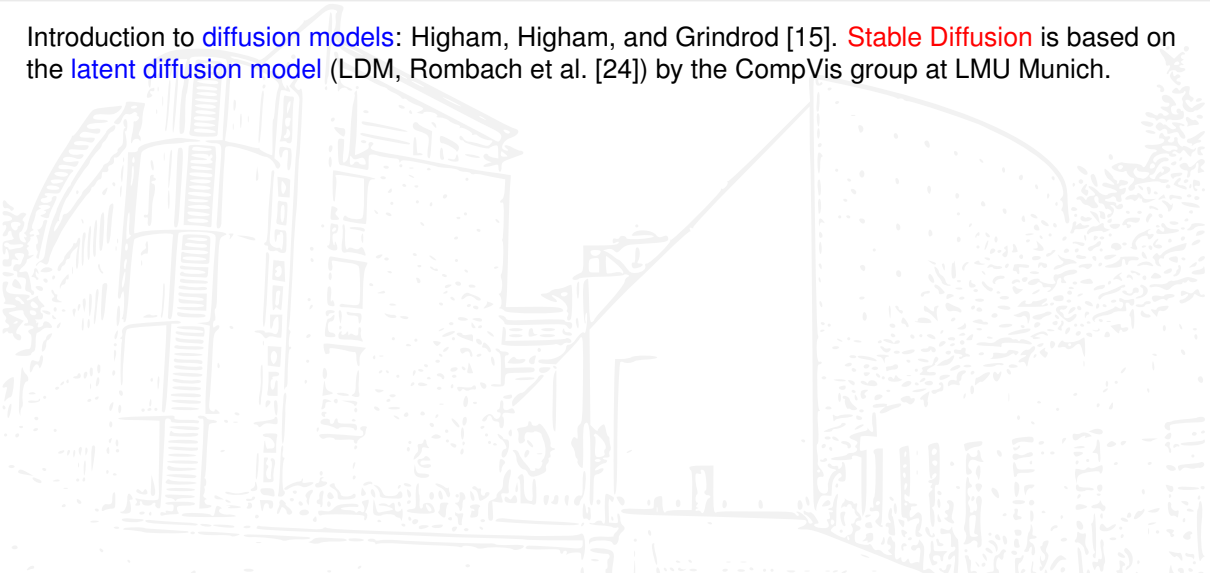
Stable Diffusion

Introduction to **diffusion models**: Higham, Higham, and Grindrod [15].



Stable Diffusion

Introduction to **diffusion models**: Higham, Higham, and Grindrod [15]. **Stable Diffusion** is based on the **latent diffusion model** (LDM, Rombach et al. [24]) by the CompVis group at LMU Munich.



Stable Diffusion

Introduction to **diffusion models**: Higham, Higham, and Grindrod [15]. **Stable Diffusion** is based on the **latent diffusion model** (LDM, Rombach et al. [24]) by the CompVis group at LMU Munich.

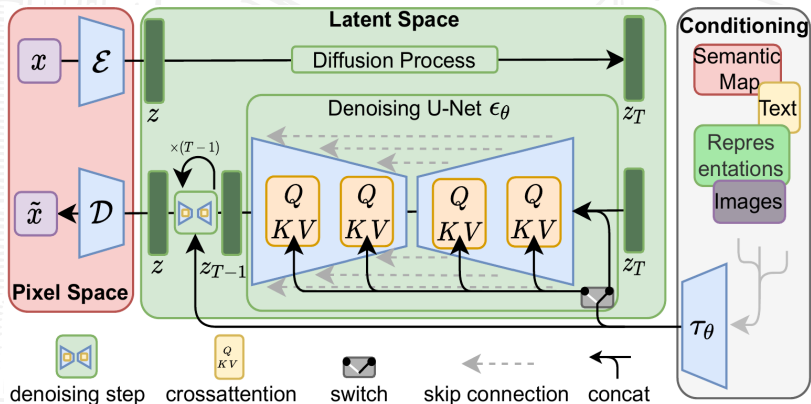


Image taken from Rombach et al. [24]

References I

- [1] Jean-Baptiste Alayrac et al. “Flamingo: a visual language model for few-shot learning”. In: *arXiv e-prints*, arXiv:2204.14198 (Apr. 2022). arXiv: 2204.14198 [cs.CV].
- [2] Martin Arjovsky, Soumith Chintala, and Léon Bottou. “Wasserstein GAN”. In: *arXiv e-prints*, arXiv:1701.07875 (Jan. 2017). arXiv: 1701.07875 [stat.ML].
- [3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. “Layer Normalization”. In: *arXiv e-prints*, arXiv:1607.06450 (July 2016). arXiv: 1607.06450 [stat.ML].
- [4] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate”. In: *arXiv e-prints*, arXiv:1409.0473 (Sept. 2014). arXiv: 1409.0473 [cs.CL].
- [5] Stella Biderman and Edward Raff. “Neural Language Models are Effective Plagiarists”. In: *arXiv e-prints*, arXiv:2201.07406 (Jan. 2022). arXiv: 2201.07406 [cs.CL].
- [6] H. Bourlard and Y. Kamp. “Auto-association by multilayer perceptrons and singular value decomposition”. In: *Biol. Cybernet.* 59.4-5 (1988), pp. 291–294. ISSN: 0340-1200. DOI: 10.1007/BF00332918.

References II

- [7] Tom B. Brown et al. “Language Models are Few-Shot Learners”. In: *arXiv e-prints*, arXiv:2005.14165 (May 2020). arXiv: 2005.14165 [cs.CL].
- [8] Sneha Chaudhari et al. “An Attentive Survey of Attention Models”. In: *ACM Transactions on Intelligent Systems and Technology* 12.5 (Oct. 2021), pp. 1–32. DOI: 10.1145/3465055.
- [9] Andrew M. Dai and Quoc V. Le. “Semi-supervised Sequence Learning”. In: *arXiv e-prints*, arXiv:1511.01432 (Nov. 2015). arXiv: 1511.01432 [cs.LG].
- [10] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *arXiv e-prints*, arXiv:1810.04805 (Oct. 2018). arXiv: 1810.04805 [cs.CL].
- [11] Alexey Dosovitskiy et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *arXiv e-prints*, arXiv:2010.11929 (Oct. 2020). arXiv: 2010.11929 [cs.CV].
- [12] Jonas Gehring et al. “Convolutional Sequence to Sequence Learning”. In: *arXiv e-prints*, arXiv:1705.03122 (May 2017). arXiv: 1705.03122 [cs.CL].

References III

- [13] Ian Goodfellow et al. “Generative Adversarial Nets”. In: *Proceedings of the International Conference on Neural Information Processing Systems (NIPS 2014)* (2014), pp. 2672–2680. URL: <https://papers.nips.cc/paper/5423-generative-adversarial-nets.pdf>.
- [14] Ishaan Gulrajani et al. “Improved Training of Wasserstein GANs”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017. URL: <https://proceedings.neurips.cc/paper/2017/file/892c3b1c6dccd52936e27cbd0ff683d6-Paper.pdf>.
- [15] Catherine F. Higham, Desmond J. Higham, and Peter Grindrod. “Diffusion Models for Generative Artificial Intelligence: An Introduction for Applied Mathematicians”. In: *arXiv e-prints*, arXiv:2312.14977 (Dec. 2023). arXiv: 2312.14977 [cs.LG].
- [16] Junnan Li et al. “Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation”. In: *arXiv e-prints*, arXiv:2201.12086 (Jan. 2022). arXiv: 2201.12086 [cs.CV].

References IV

- [17] Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. “Effective Approaches to Attention-based Neural Machine Translation”. In: *arXiv e-prints*, arXiv:1508.04025 (Aug. 2015). arXiv: 1508.04025 [cs.CL].
- [18] Volodymyr Mnih et al. “Recurrent Models of Visual Attention”. In: *Advances in Neural Information Processing Systems*. Ed. by Z. Ghahramani et al. Vol. 27. Curran Associates, Inc., 2014. URL: <https://proceedings.neurips.cc/paper/2014/file/09c6c3783b4a70054da74f2538ed47c6-Paper.pdf>.
- [19] OpenAI. “GPT-4 Technical Report”. In: *arXiv e-prints*, arXiv:2303.08774 (Mar. 2023). arXiv: 2303.08774 [cs.CL].
- [20] Alec Radford et al. *Improving Language Understanding by Generative Pre-Training*. OpenAI Blog, <https://openai.com/blog/language-unsupervised/>. June 2018. URL: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

References V

- [21] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. OpenAI Blog, <https://openai.com/blog/better-language-models/>. Feb. 2019. URL: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
- [22] Alec Radford et al. “Learning Transferable Visual Models From Natural Language Supervision”. In: *arXiv e-prints*, arXiv:2103.00020 (Feb. 2021). arXiv: 2103.00020 [cs.CV].
- [23] Anna Rogers, Olga Kovaleva, and Anna Rumshisky. “A Primer in BERTology: What We Know About How BERT Works”. In: *Transactions of the Association for Computational Linguistics* 8 (2020), pp. 842–866. DOI: 10.1162/tac1_a_00349. URL: <https://aclanthology.org/2020.tac1-1.54>.
- [24] Robin Rombach et al. “High-Resolution Image Synthesis with Latent Diffusion Models”. In: *arXiv e-prints*, arXiv:2112.10752 (June 2022). International Conference on Computer Vision and Pattern Recognition (CVPR). New Orleans, LA. pp. 10684–10695. arXiv: 2112.10752 [cs.CV].

References VI

- [25] Alana de Santana Correia and Esther Luna Colombini. “Attention, please! A survey of Neural Attention Models in Deep Learning”. In: *arXiv e-prints*, arXiv:2103.16775 (Mar. 2021). arXiv: 2103.16775 [cs.LG].
- [26] Ian Tenney, Dipanjan Das, and Ellie Pavlick. “BERT Rediscovered the Classical NLP Pipeline”. In: *arXiv e-prints*, arXiv:1905.05950 (May 2019). arXiv: 1905.05950 [cs.CL].
- [27] Ashish Vaswani et al. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017. URL: <https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>.
- [28] Jason Weston, Sumit Chopra, and Antoine Bordes. “Memory Networks”. In: *arXiv e-prints*, arXiv:1410.3916 (Oct. 2014). arXiv: 1410.3916 [cs.AI].

- [29] Kelvin Xu et al. “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention”. In: *Proceedings of the 32nd International Conference on Machine Learning*. Ed. by Francis Bach and David Blei. Vol. 37. Proceedings of Machine Learning Research. Lille, France: PMLR, July 2015, pp. 2048–2057. URL: <https://proceedings.mlr.press/v37/xuc15.html>.